

**SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM
CSÍKSZEREDAI KAR
GAZDASÁGI INFORMATIKA SZAK**

DIPLOMADOLGOZAT

**Végzős hallgató:
Farkas Szabolcs**

**Témavezető:
Dr. Garda-Mátyás Edit, egyetemi adjunktus**

2026

**SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM
CSÍKSZEREDAI KAR
GAZDASÁGI INFORMATIKA SZAK**

DIPLOMADOLGOZAT

**LINEÁRIS ALGEBRAI PROBLÉMÁK
SZÁMÍTÓGÉPES PREZENTÁLÁSA**

**Végzős hallgató:
Farkas Szabolcs**

**Témavezető:
Dr. Garda-Mátyás Edit, egyetemi adjunktus**

2026

Kivonat

Szakedolgozatom témája lineáris algebrai problémák bemutatása egy olyan számítógépes környezetben, amely egyszerűen és könnyen érthetően prezentálja a matematika sokoldalúságát, valamint felhasználhatóságát.

Az alkalmazás célja, hogy egy egyszerű kezelőfelület segítségével minél változatosabban és látványosabban értesse meg a felhasználóval a kiválasztott lineáris algebrai problémát, ezzel is elősegítve, hogy az embereket közelebb hozza ehhez a tudományághoz, támogatva a hatékonyabb tanulást és a gyakorlati megoldások kipróbálását.

Az alkalmazás kezelőfelülete roppant egyszerű, a problémák kategóriákra vannak bontva, a felhasználó pedig könnyen tud tájékozódni, valamint ezeket a menüpontokat kiválasztva a kategóriában lévő feladatok közül tud választani.

Minden problémafelület rendelkezik egy teljesen véletlenszerűen generált érték móddal, ahol az alapparaméterek megadása után a rendszer automatikusan feltölti a feladatot ezekkel az értékekkel, amit a felhasználó a megoldás előtt tetszés szerint módosíthat. Rendelkezik egy üres mezőkkel rendelkező felülettel is, ahol nincsen jelen semmilyen előre generált érték, ennek a célja az, hogy átláthatóbb legyen a feladat kitöltése. Valamint jelen van egy ismertető fül is, ahol a felhasználó elolvashatja az adott probléma szabályait és működésrendjét.

Akadálymentesen lehet alkalmazni az oktatásban, ahol a tanárok órán nemcsak a táblán felírt képletek vagy ábrák segítségével tudnak bemutatni egy feladatot, hanem egy olyan dinamikus vizuális ábrát is adhat, amit anélkül lehet módosítani, hogy az egész feladatot, vagy csak egy részét manuálisan újra kellene számolni és ahol megfigyelhető, ahogyan egy diagram vagy feladat eredménye ezáltal változik. A diákok akár önállóan, a tanár jelenléte nélkül is megérthetik és gyakorolhatják ezeket a problémákat, ezzel is élvezhetőbbé tenni a tanulást és barátságosabbá tenni a matematikát.

A projekt megvalósítása során lehetőségem nyílt mélyebben megismerkedni a lineáris algebrai problémák szabályaival, működésrendjével, ezek felhasználásával a mindennapjainkban, valamint az ablakos alkalmazások elkészítésének technológiáival. Sikeresen kidolgoznom egy olyan megoldást, ami elősegíthet más embereket is közelebb hozni a matematikához és informatikához egyaránt.

Rezumat

Tema lucrării mele de licență este prezentarea problemelor din algebra liniară într-un mediu informatic, care ilustrează într-un mod simplu și ușor de înțeles versatilitatea și utilitatea Tema lucrării mele de licență este prezentarea unor probleme de algebră liniară într-un mediu informatic care ilustrează, într-un mod simplu și ușor de înțeles, versatilitatea și utilitatea matematicii.

Scopul aplicației este de a ajuta utilizatorul să înțeleagă problema de algebră liniară selectată într-un mod cât mai variat și mai spectaculos, cu ajutorul unei interfețe simple, contribuind astfel la apropierea oamenilor de această disciplină științifică și sprijinind învățarea mai eficientă și testarea soluțiilor practice.

Interfața aplicației este extrem de simplă, problemele fiind împărțite pe categorii, iar utilizatorul se poate orienta cu ușurință; de asemenea, selectând aceste opțiuni din meniu, acesta poate alege dintre exercițiile din categoria respectivă.

Fiecare pagină de exerciții dispune de un mod de generare complet aleatorie a valorilor, în care, după introducerea parametrilor de bază, sistemul completează automat exercițiul cu aceste valori, pe care utilizatorul le poate modifica după bunul plac înainte de a găsi soluția. Există, de asemenea, o interfață cu câmpuri goale, în care nu sunt prezente valori generate în prealabil, scopul acesteia fiind acela de a face completarea exercițiului mai transparentă. De asemenea, este prezentă o filă de prezentare, unde utilizatorul poate citi regulile și modul de funcționare al problemei respective.

Poate fi utilizat fără restricții în domeniul educației, unde profesorii pot prezenta o problemă în timpul orei nu doar cu ajutorul formulelor sau diagramelor scrise pe tablă, ci pot oferi și o reprezentare vizuală dinamică, care poate fi modificată fără a fi necesar să se recalculeze manual întreaga problemă, sau doar o parte a acesteia, și unde se poate observa cum se modifică astfel rezultatul unui diagramă sau al unei exerciții. Elevii pot înțelege și exersa aceste probleme chiar și în mod independent, fără prezența profesorului, făcând astfel învățarea mai plăcută și matematica mai accesibilă.

Pe parcursul realizării proiectului, am avut ocazia să mă familiarizez mai în profunzime cu regulile și modul de funcționare al problemelor de algebră liniară, cu utilizarea acestora în viața de zi cu zi, precum și cu tehnologiile de creare a aplicațiilor cu ferestre. Am reușit să elaborez o soluție care poate contribui la apropierea altor persoane atât de matematică, cât și de informatică.

Abstract

The topic of my thesis is the presentation of linear algebra problems in a computer-based environment that illustrates the versatility and practical applications of mathematics in a simple and easy-to-understand way.

The goal of the application is to help users understand the selected linear algebra problem in as varied and engaging a way as possible through a simple user interface, thereby bringing people closer to this field of study and supporting more effective learning and the testing of practical solutions.

The app's interface is extremely simple; problems are organized into categories, allowing users to easily navigate the app and select from the exercises within each category by choosing the appropriate menu items.

Each problem screen features a fully random-value mode: after the user enters the basic parameters, the system automatically populates the problem with these values, which the user can modify as desired before solving it. There is also an interface with blank fields, where no pre-generated values are present; the purpose of this is to make filling out the problem more transparent. Additionally, there is a "Help" tab where the user can read the rules and operating procedures for the given problem.

It can be used seamlessly in education, where teachers can not only present a problem using formulas or diagrams written on the board, but can also provide a dynamic visual representation that can be modified without having to manually recalculate the entire problem, or just a part of it, and where students can observe how the result of a diagram or problem changes as a result. Students can understand and practice these problems on their own, even without the teacher present, making learning more enjoyable and mathematics more approachable.

While working on this project, I had the opportunity to gain a deeper understanding of the rules and mechanics of linear algebra problems, their applications in our daily lives, and the technologies involved in developing window-based applications. I was able to develop a solution that can help bring other people closer to both mathematics and computer science.

A mesterséges intelligencia használata

Alulírott *Farkas Szabolcs* kijelentem, hogy a jelen *Lineáris algebrai problémák számítógépes prezentálása* című dolgozatom és az alkalmazás elkészítése során generatív mesterséges intelligencia eszközt az alábbi módon vettem igénybe:

- Gemini 3.1: Bonyolultabb képletek egyszerűsítése és magyarázata megértéshez (5.fejezet).

Tudomásul veszem, hogy az MI által generált tartalomért teljes mértékben felelősséggel tartozom, és tisztában vagyok azzal, hogy az MI nem megfelelő vagy jelöletlen használata a Sapientia EMTE vonatkozó szabályzatai szerinti következményekkel járhat.

Tartalomjegyzék

1.	Bevezető	9
2.	Elméleti megalapozás	10
2.1.	Szakirodalmi áttekintő	10
2.2.	Hasonló alkalmazások	11
3.	A rendszer specifikációja	13
3.1.	Felhasználói követelmények.....	13
3.2.	Rendszerkövetelmények	13
3.2.1.	Funkcionális követelmények.....	13
3.2.2.	Nem funkcionális követelmények.....	14
4.	Tervezés és megvalósítás	15
4.1.	Használt technológiák.....	15
4.2.	Az alkalmazás architektúrája	16
5.	Alkalmazás bemutatása	17
5.1.	Rendszer előkészítése indításkor	17
5.2.	Ablakok felépítése	18
5.3.	Mátrix problémák bemutatása	19
5.3.1.	Mátrixok összeadása	20
5.3.2.	Mátrixok szorzása	21
5.3.3.	Transzponált mátrix.....	21
5.3.4.	Egységmátrix.....	22
5.3.5.	Inverz mátrix	23
5.3.6.	Mátrix determinánsa	24
5.3.7.	Háromszögmátrix	24
5.3.8.	Cramer-szabály.....	26
5.4.	Vektor problémák bemutatása	27
5.4.1.	Vektorok normája	28
5.4.2.	Vektorok összeadása	29
5.4.3.	Skalárral való szorzás	30
5.4.4.	Lineáris kombináció	31
5.4.5.	Két vektor skaláris szorzata	33
5.4.6.	Vektorok vektoriális szorzata.....	34
5.4.7.	Háromszög megoldás.....	35
5.5.	Vektorterek bemutatása	37

5.5.1.	Alterek	39
5.5.2.	Lineáris függetlenség	40
5.5.3.	Bázis	41
5.5.4.	Bázis Transzformáció	43
5.5.5.	Mátrix rangja	44
5.5.6.	Lineáris egyenletrendszerek	46
5.6.	Lineáris algebra a mindennapokban	48
5.6.1.	Digitális képfeldolgozás	48
5.6.2.	Lineáris programozás	52
5.6.3.	Portfólió optimalizálás	54
6.	Következtetések	56
	Irodalomjegyzék	57

1. Bevezető

A lineáris algebra a mindennapjaink részét képezi, valamint jelen van a digitális világunkban is, ezért kiemelten fontos, hogy a diákok jól megismerjék és elsajátítsák ezt a területet. A hagyományos oktatásban a tanár az osztályteremben felír egy lineáris egyenletrendszert, megoldja, megmutatja, hogy és miért változik az ismeretlen, de minden értékmódosításnál újra kell írni az egész számítást, leellenőrizni, és utána pedig összehasonlítani a két számítás közötti értéket. A digitális világ megszületése óta olyan erős számítás segítő eszközeink vannak mint a számítógépek, ezek jelentősen segítik és gyorsítják a mindennapi teendőinket, beleértve a komplex matematikai műveleteket is.

Erre a szemléltetési és oktatási kihívásra kínál megoldást az alkalmazás, amellyel a különböző problémákat gyorsan, egyszerűen, és látványosan be lehet mutatni az érdeklődők számára. A cél egy olyan digitális interfész létrehozása amellyel a diákokat és tanárokat egyaránt lehet segíteni a lineáris algebra szépségének a bemutatására valamint arra, hogy miként befolyásol egy érték egy eredményt, ezt pedig látványosan és gyorsan megjeleníteni, ezzel is fenntartani az érdeklődést a mai felgyorsult világban.

A lineáris algebra kifejezetten jelentős a gazdasági életben is (kockázatszámítás, optimalizálás stb.), valamint a digitális világunkban, ami egyre jobban fejlődik és terjeszkedik már-már megállíthatatlanul, például a képfeldolgozás, ahogyan egy kép rendelkezik pixelekkkel, és a pixelek rendelkeznek bizonyos színnel, ezek mind mátrix feladatok, amit a számítógépünk gyorsan és egyszerűen meg tud jeleníteni és módosítani.

Az alkalmazás letölthető és kipróbálható a következő GitHub linkről:
<https://github.com/farkasszabolcs-sap/Farkas-Szabolcs-Szakdolgozat>

2. Elméleti megalapozás

A dolgozat következő fejezeteiben részletesen bemutatom azokat a technológiákat, amelyeket az alkalmazásom fejlesztése közben használtam.

2.1. Szakirodalmi áttekintő

Windows: A Windows egy számítógépes operációs rendszer, amit a Microsoft Corporation fejlesztett ki, rendelkezik beépített grafikus felülettel. Szabványos felületet nyújtanak, amelyek menükre, ablakokra, valamint egy mutatóeszköze, például egy egérre alapszik. Alapfilozófiája a popularitás, a könnyen kezelhetőség, valamint a “minden egyben” filozófia.

Python: A Python egy általános célú, nagyon magas szintű programozási nyelv. A nyelv filozófiája az olvashatóságot és a programozói munka megkönnyítését helyezi előtérbe a futási sebességgel szemben. Úgynevezett interpreteres nyelv, ami azt jelenti, hogy a forráskód és a tárgykód nincs különválasztva, és máris futtatható a megírt program. Az egyik legnépszerűbb programozási nyelv. Nyitott, közösségalapú fejlesztési modellel rendelkezik, amit a Python Software Foundation felügyel. (Python dokumentáció, 2025)

Python fejlesztési eszközök: A PyCharm egy integrált fejlesztői környezet (IDE), amelyet a Pythonban való programozáshoz használnak. Kódelemzést, grafikus debuggerrel, valamint verzióirányító rendszerrel rendelkezik. A PyCharm úgynevezett cross-platform rendszer, hiszen működik Windows-on, macOS-en, és Linux-on egyaránt. (Pycharm dokumentáció, 2025)

Python csomagok: A fejlesztés során számos csomagot használtam ami megkönnyítette a fejlesztés különböző területeit: Numpy (lineáris algebrai műveletek elvégzése), Scipy (lineáris programozás), Tkinter (ablak alkalmazás alapja), Matplotlib (Diagramok elkészítése), Random (véletlenszerűen generált számok létrehozása), Importlib (modulok importálása), Pathlib (fájlok elérhetőségi útvonalainak kezelése), PIL (képekkel kapcsolatos műveletek)

Lineáris algebra: A lineáris algebra (Puskás, Szabó, & Tallos, 1997) eredetileg lineáris egyenletrendszerek megoldásával foglalkozott, ezért először csak a mátrixaritmetika és determinánselmélet tartozott a tárgyköréhez. A matematika ezen tudományága, jelentős geometriai, fizikai, és mérnöki alkalmazással rendelkezik. A modern közgazdaságtudományban is jelentős fontossággal bír. Mátrixok, vektorok, vektorterek, lineáris egyenletrendszerek tartoznak hozzá, de rendkívül fontos szerepe van a modern számítástechnika területén is. Roppant egyszerűen meglehet oldani a lineáris

egyenletrendszereket, átalakítjuk egy mátrixra, aztán pedig azt kiszámoljuk, de egy modern képfeldolgozás is lineáris algebra segítségével történik, hiszen egy kép egy hatalmas mátrix, minden benne szereplő elem egy pixelt jelképez, és ezek a pixelek egy bizonyos szint tárolnak. A 2D-s és 3D-s transzformációk mátrixokkal történő kiszámítása fontos szerepet töltenek be egy számítógépes grafikában, ahol az adott alakzatot, forgatjuk, méretezzük különböző lineáris algebrai műveletekkel. Hatalmas mennyiségű adathalmazok hatékonyan való kezelése és elemzése is mátrixokkal való műveletek. Közgazdaságtanban is például gazdasági modelleknél, portfólió-optimalizációnál jelentős szerepet tölt be.

2.2. Hasonló alkalmazások

Az alkalmazás fejlesztése során kiemelten fontos szempont volt, hogy a piacon jelen levő, hasonló alkalmazások funkcióit is figyelembe vegyem. Az alábbiakban bemutatok pár nemzetközi alkalmazást, amelyeknek a célja szintén matematikai problémák bemutatása vagy kiszámítása egy digitális környezetben.

Symbolab: Egy weboldalas alkalmazás, rendelkezik különböző matematikai számológépekkel, képes függvényszámításra, geometriai és lineáris számolásokra szintúgy. Nagyon erős, részletes és kiterjedt példatárral rendelkezik, rengeteg feladattal. Funkcionális szempontból érdemes kiemelni, hogy rendelkezik egy csoport létrehozási rendszerrel, ahol a tanárok akár quiz-t csinálhatnak a diákjaiknak, ezzel is egyfajta játékosabb és látványos képességfelmérést szervezni az óra keretein belül. (Symbolab, 2026)

GeoGebra: Rendelkezik weboldalas alkalmazással, számítógépes programmal, valamint mobilokon is elérhető eszközökkel. A felhasználó kedve szerint választhat 2D, 3D geometriát, esetleg valószínűségszámítást, vagy komplex függvényszámítást. Rendkívül egyszerű és széleskörben elterjedt. Funkcionális téren nagyon fontos kiemelni, hogy rendelkezik egy interaktív tanító példakönyvtárral, ahol 4-12 osztályban megtanult különböző matematikai feladatokat meg lehet tanulni, elsajátítani, valamint ezek az interaktív példák valós időben megmutatják a változásokat, eredményeket, és visszakérdeznek, ezzel is hatékonyan és egyszerűen tanulva. (Geogebra, 2026)

Maplesoft: A Maplesoft egy modern és innovatív megoldást kínál a mai kihívásokhoz, matematikai problémák mobileszközökön való vizsgálatától kezdve, egészen a fejlesztési kockázatok csökkentéséig egy komplex mérnöki projektben. Több mint 8000 oktatási intézményben, kutató laborokban és egyéb cégeknél használják több mint 90 országban. Mérnököktől elkezdve, kutatókig, és diákokig mindenki megtalálja a számára kellő eszközt,

ami segítheti munkájában, esetleg tanulásában. Különböző verzióval rendelkezik, ahol jelen van a tanulóknak szánt edukatív jelleggel készült program, valamint mérnököknek, kutatóknak szánt erős és komplex rendszer, egészen az egyszerű számolásoktól kezdve, adatelemzésekig, algoritmusfejlesztésig, vagy akár különböző modellek például atomok modellezésig, az alkalmazás kellően fel van szerelve eszközökkel. Egy ideális matematikai alkalmazás függetlenül, hogy tanuló, mérnök, kutató, vagy egyetemista az illető, viszont az előfizetési rendszer miatt az alkalmazás nehezen elérhető és kipróbálható a 15 napos próbaidőszakon kívül.

A fent említett alkalmazások jól mutatják, hogy különböző matematikai problémákat látványosan meglehet mutatni, valamint jól megmagyarázni, viszont a könnyen és ingyen elérhető alkalmazások közül jelentősen kevés pillant bele a lineáris algebra részleteibe és problémáiba, és ott is csak néhány egyszerűbb példára fókuszál. Az alkalmazásom ezekkel szemben viszont jóval részletesebben, több problémát prezentál, valamint bemutat pár mindennapi életben használt lineáris algebrai felhasználást is, ami jóval érdekesebbé teheti a matematikát a diákok számára.

3. A rendszer specifikációja

Az alkalmazásom lineáris algebrai problémákat bemutató alkalmazás számítógépes környezetben, amelynek célja, hogy ezeket a problémákat látványosan, egyszerűen és könnyen megérthetően prezentálja az eziránt érdeklődők felé. Az alkalmazás felhasználói lehetnek tanárok, diákok, vagy egyszerű kíváncsi emberek. A funkciók bármilyen bejelentkezés, regisztráció vagy előfizetés nélkül egyszerűen elérhetőek, egy probléma navigáló menün belül pedig kiválaszthatják a nekik megfelelő opciót.

3.1. Felhasználói követelmények

A felhasználó egy probléma navigációs mezőn belül kiválaszthatja a lineáris algebrai probléma típusát, ez lehet mátrix, vektor, vektortér, esetleg mindennapi felhasználás, ezeken belül pedig részletesebben fel vannak bontva a problémák, ebből amint a felhasználó kiválaszt egyet, megjelennek a problémának megfelelő beviteli mezők, a program megkérdezi például, hogy mekkora mátrixokat szeretne, vagy hány ismeretlennel szeretne számolni. Ezt kétféle módon teheti meg, van egy fül saját értékek megadására, vagy előre generált értékek megadására.

Helyesen kitöltve a legenerált mezőket, ezután a felhasználó kiszámoltathatja a problémától függően az eredményt, esetleg megjeleníthet diagramokat, ábrákat. Módosítás után a felhasználó egy gombnyomással kiszámolhatja az új eredményt, és a program kiszámolja és megjeleníti az új eredményt, ezáltal is akadálymentesen prezentálja, hogy miként változik az eredmény, ha akár csak egy értéket módosítunk.

3.2. Rendszerkövetelmények

A program működéséhez elengedhetetlen egy asztali számítógép vagy laptop, Python 3.11-es verzió letöltése, valamint minimum Windows 7-es operációs rendszer, és 4GB RAM az akadálymentes működéshez. A program nem használ internetkapcsolatot, ezért a kellő előkészületek után bármikor és bárhol a felhasználó futtathatja és használhatja bármilyen probléma nélkül.

3.2.1. Funkcionális követelmények

- Műveletek pontos megoldása
- A kellő eredmények helyes megjelenítése

- Diagrammok, ábrák pontos ábrázolása
- Dinamikus méret generálás felhasználók igénye szerint
- Üres mezőkkel rendelkező probléma generálás
- Előre generált értékekkel rendelkező probléma generálás
- Képszerkesztésnél kép feltöltése, szerkesztése, és kimentése
- Problémáknak, példáknak a működési leírásának megtekintése

3.2.2. Nem funkcionális követelmények

- Felhasználóbarát grafikus felület, egyszerű navigáció
- Problémák kategóriába való sorolása, és menüpont mögé helyezése
- Gyors és akadálymentes működés

A program kidolgozásánál fontos cél volt, hogy a felhasználó a sok, akár számára elsőre ismeretlen problémák és példák között is egyszerűen megtalálja mit keres, meg tudja tekinteni, és ki tudja próbálni. Mindegyik menüpont csak a kiválasztott problémával foglalkozik, ezzel is elkerülve a felesleges plusz információkat mindegyik példa megmagyarázásánál, bemutatásánál.

4. Tervezés és megvalósítás

Az alábbi fejezetekben beszélni fogok a lineáris algebrai problémák számítógépes prezentálásának a fejlesztési folyamatáról, valamint a legfontosabb lépéseiről, elméleti háttereiről. A hangsúlyt az elmélet és a gyakorlat kapcsolatára, a választott technológiák indoklására, valamint az alkalmazás architektúrájának az ismertetésére helyezem.

4.1. Használt technológiák

A program fejlesztése során több megbízható és modern technológiát használtam, amely elősegíti a lineáris algebra iránt érdeklődőknek a problémák helyes megoldását, bemutatását, és gyors használatát.

- **Windows operációs rendszer:** A program célplatformja a Windows, amelyet Python programozási nyelv használatával valósítottam meg. A PyCharm fejlesztőkörnyezet lehetővé tette számomra hogy az architektúra könnyen átlátható legyen, problémákat egyszerűbben kijavítsak, valamint az akadálymentes Python csomagok telepítését, hiszen számos csomagot használatba vettem a fejlesztés során, amely mint felhasználó mint fejlesztési téren megkönnyítik a program kivitelezését.
- **Python:** A program teljes része Python 3.11-ben lett megírva. Biztosítja az egyszerű fejlesztést, könnyű karbantarthatóságot, és az egyik legnépszerűbb programozási nyelv. A felhasznált csomagok és modulok:
- **Numpy:** A matematikai műveletek elvégzéséhez elengedhetetlen eszköz, egyszerű használni, és pontos eredményeket kínál.
- **Tkinter:** A program ablakos megjelenítéséhez alap eszköz volt, amely lehetővé teszi, hogy gombok, beviteli mezők, és egyéb eszközök segítségével egyszerűen be tudjuk táplálni a feladatok adatait, valamint átlátható, formázható, és felhasználóbarát környezetben lehessen a problémákkal dolgozni.
- **Matplotlib:** A 3D, 2D, oszlopdiagrammok elkészítéséhez rendkívül fontos eszköz volt, hiszen sok lineáris algebrai problémát akkor lehet látványosan bemutatni, és ábrázolni, ha ezeket vizuálisan megjelenítjük, hogy az eredmény, esetleg az értékmódosítások átláthatóak legyenek.
- **Scipy:** Lineáris programozás megoldásához használtam, melyet a kellő értékekkel, és határértékekkel megoldhatunk, és később ábrázolhatunk.

- **Random:** Véletlenszerűen generált értékek létrehozása jelentősen leegyszerűsíti a problémák értékekkel való feltöltését.
- **Importlib:** Dinamikus modul importálást használtam az alkalmazás futtatásakor, amely minden ablakot dinamikusan betölt, ellenőriz, így akadálymentesen lehet váltani menüpontok között használatkor.
- **Pathlib:** Olyan modulokhoz amelyek külön fájlokban helyezkednek el, az elérési útvonaluk elengedhetetlen egy dinamikus betöltéshez.
- **PIL:** Digitális képfeldolgozás megvalósításánál, hogy egy képet feltöltés után módosítani tudjunk lineáris algebrai módszerekkel, majd pedig azt megjelenítsünk.

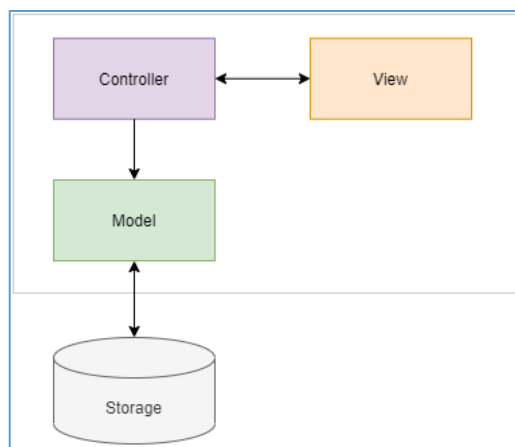
4.2. Az alkalmazás architektúrája

Az alkalmazás egy MVC (MVC dokumentáció, 2025) architektúrát követ, amely elősegíti a kód jól strukturált, valamint jól karbantartható felépítését. Az alkalmazás felépítését három részre lehet felbontani, Model, View, és Controller részekre, ezt láthatjuk a 4.1. ábrán.

Az MVC modellben a Model felelős a háttérszámításokért, adatbáziskezelésekért, esetünkben a háttérszámításokért felelős.

A View felelős a felhasználói felületért. Ezt megvalósítja a Tkinter mint grafikus keretrendszer, amely lehetővé teszi, hogy a felhasználó interakcióba lépjen az alkalmazással, és annak beépített funkcióival.

A Controller felelős a Model és a View közötti interakcióért, fenntartásáért, valamint a felhasználói felületek váltakozásáért. Fontos megemlíteni, hogy a Model és View közvetlenül egymással nem kommunikálhat, ezek csak a Controllerek segítségével léphetnek kapcsolatba, rajtuk keresztül kerül küldésre és fogadásra az adat.



4.1. ábra. Az alkalmazás MVC architektúrája
(Forrás: <https://www.pythontutorial.net/tkinter/tkinter-mvc/>)

5. Az alkalmazás bemutatása

Mivel az alkalmazás a lineáris algebrai problémákon alapszik, ezért kiemelt hangsúlyt fektettem ezen elméletek és tulajdonságok megértésére. Implementálja a legfontosabb mátrix- és vektorműveleteket, bemutatja a vektorterek tulajdonságait, valamint mindennapokban használt példákat is feldolgoz. Az alábbi fejezetekben beszélni fogok ezekről a problémákról, tulajdonságaikról, hogyan oldottam meg ezeket a gyakorlatban, valamint az alkalmazás működéséről.

5.1. Rendszer előkészítése indításkor

Az alkalmazás indításakor automatikusan betöltésre kerül a stílusfájl, hiszen alkalmaztam a *Themed Tkintert*, ami a *Tkinter* egy modifikálható változata, amelynek segítségével font, betűméret, szín, gombstílus, és egyéb kozmetikai beállításokat lehet változtatni. Ezek mellett betöltésre kerül az összes ablak amik között a felhasználó képes lesz navigálni. Ezt az *Importlib* és *Pathlib* segítségével értem el, ahol fájlanként ellenőrizve az osztályok neveit dinamikusan importáljuk későbbi használatra.

Először a “views” mappában amiben ezen ablakok vannak elhelyezve, a program megnézi azokat amik nincsenek mappában, tehát a főbb ablakok fájljai kerülnek regisztrálásra. Ezt követve a problémák ablakait, amik külön kategorizálva vannak mappákban. Ezek regisztrálása fájlról fájlra történik, ahol ellenőrizzük a benne szereplő osztály nevét a fájl nevével.

Miután az ellenőrzés megtörtént, az osztályt ki importáljuk egy külön *screen_class* változóba, ami átkerül a *ScreenController*-en belül egy “*self.screens[]*” listába. Ezzel is felgyorsítva a gyors ablakváltást, és az egyszerűbb karbantartást elősegítve. A kategorizált probléma betöltést egy erre specifikált *auto_register_problems()* metódus végzi.

```
def auto_register_screens(self):
    print(f"-->Alap menük betöltése")
    view_dir = "views"
    view_path = Path(view_dir)
    for file_path in view_path.glob("*.py"):
        module_name = file_path.stem
        class_name = module_name.capitalize()
        try:
            module = importlib.import_module(f"{view_dir}.{module_name}")
            if (hasattr(module, class_name)):
                screen_class = getattr(module, class_name)
                self.register_screen(screen_class)
        except Exception as e:
            print(f"\033[91m-> {class_name} sikertelenül betöltve: {e}\033[0m")
```

5.1. ábra. Részlet az *ApplicationController* működéséről – *auto_register_screens()* metódus

Ezen említett fájlok betöltése előtt létrejön az ablakok alapja, amire épül az egész ablakrendszer, a *Tkinter root*, amire később ezek az ablakok kerülnek, hiszen nem szeretnénk többretegű ablakokat, hanem azt szeretnénk, hogy minden egy ablakban történhessen.

A *ScreenController* nemcsak az ablakok regisztrálásáért felelős, hanem ezek cserélésére is navigáció történésekor. Ablacímét cserél, majd pedig egyszerűen a meglévő ablakot eltünteti, meghívja az ablakot az előre beregisztrált ablakokból, ezt egyszerűen csak megjeleníti.

```
#ablakok cserélése navigációkor ablaknév alapján
def show_screen(self, screen_name):

    self.change_title(screen_name)

    if self.current_screen:
        self.current_screen.grid_remove()

    screen = self.screens[screen_name]
    screen.grid()
    self.current_screen = screen
```

5.2. ábra. Részlet a *ScreenController* működéséről – *show_screen()* metódus

Ez a dinamikus rendszer lehetővé teszi, hogy az alkalmazást könnyen bővítsük új ablakokkal, átlátható maradjon, és egyszerűen karbantartható maradjon. (5.1. ábra)

```
# ablak megjelenítése megnyitáskor
def create_screen(self):
    self.matrix_menu=ttk.Frame(self)
    ttk.Label(self, text="Mátrix műveletek", style='Title.TLabel').grid(row=0, column=0)
    ttk.Button(self.matrix_menu, text="Összeadás", width=30, command=lambda:self.controller.show_screen("Matrixsum")).grid(row=2, column=0)
    ttk.Button(self.matrix_menu, text="Szorzás", width=30, command=lambda:self.controller.show_screen("Matrixmult")).grid(row=3, column=0)
    ttk.Button(self.matrix_menu, text="Transzponált mátrix", width=30, command=lambda:self.controller.show_screen("Matrixtranspose")).grid(row=4, column=0)
    ttk.Button(self.matrix_menu, text="Egység mátrix", width=30, command=lambda:self.controller.show_screen("Matrixidentity")).grid(row=5, column=0)
    ttk.Button(self.matrix_menu, text="Inverz mátrix", width=30, command=lambda:self.controller.show_screen("Matrixinvert")).grid(row=6, column=0)
    ttk.Button(self.matrix_menu, text="Determináns", width=30, command=lambda:self.controller.show_screen("Matrixdeterminant")).grid(row=7, column=0)
    ttk.Button(self.matrix_menu, text="Háromszög mátrix", width=30, command=lambda:self.controller.show_screen("Matrixtriangle")).grid(row=8, column=0)
    ttk.Button(self.matrix_menu, text="Cramer szabály", width=30, command=lambda:self.controller.show_screen("Matrixcramer")).grid(row=9, column=0)
    (ttk.Button(self.matrix_menu, text="Vissza a problémákhoz", width=30, command=lambda: self.controller.show_screen("Problemsmenu"), style='Back.TButton')
     .grid(row=10, column=0))

#menü elemek középre illesztése
self.columnconfigure( index= 0, weight=1)
self.rowconfigure( index= 1, weight=1)
self.matrix_menu.grid(row=1, column=0)
```

5.3. ábra. Részlet a *Matrixmenu* működéséről – *create_screen()* metódus

5.2. Ablakok felépítése

Egy ablakokon működő alkalmazás rendkívül fontos aspektusa, hogy ha navigálunk, akkor ne új ablakok nyíljanak meg, hanem minden ugyanabban az ablakban jelenjen meg, esetleg más menü jelenjen meg helyette. Minden ablak örökli az alkalmazás elindításakor létrehozott *root*-

ot, ezzel teljesen elkerülve azt, hogy új, egymásra felugró ablakok jelenjenek meg, rombolva a felhasználói élményt, és lassítva a teljesítményt.

Minden ablak meghíváskor, tehát amikor a felhasználó megnyitja az adott menüpontot, problémát, lefut a `create_screen()` metódus, ami a menüpontoknál létrehozza az adott menüt, problémáknál létrehozza a kezdő paramétereket, *frame*-eket amikbe később beviteli mezők kerülnek, fülekkel való navigációs panelt, ami elkülöníti a szabályokat, a generáláskor üres értékekkel rendelkező beviteli mezőket, valamint a véletlenszerűen generált előre feltöltött beviteli mezőket.

```
#Mátrix összeadás kezdőképernyője
def create_screen(self):
    title_frame = tk.Frame(self)
    ttk.Label(title_frame, text="Mátrix összeadása", style='Title.TLabel').grid(row=0, column=0)
    ttk.Button(self, text="Vissza", style='Back.TButton',
               command=lambda: self.screen_controller.show_screen("Matrixmenu")).grid(row=1, column=0, sticky="w")
    title_frame.grid(row=0, column=0)

    #tabcontrol létrehozása a generálási módszerek, valamint a tulajdonságok elkülönítéséhez
    self.tabcontrol = ttk.Notebook(self)
    self.tabcontrol.grid(row=2, column=0)

    #frame-k létrehozása a két generálási módszerhez
    self.pelda_adatok = ttk.Frame(self.tabcontrol)
    self.sajat_adatok = ttk.Frame(self.tabcontrol)
    self.tulajdonsagok = ttk.Frame(self.tabcontrol)

    #tabcontrol-hoz hozzáadása ezeknek a frameeknek
    self.tabcontrol.add(self.pelda_adatok, text="Példa adatok")
    self.tabcontrol.add(self.sajat_adatok, text="Saját adatok")
    self.tabcontrol.add(self.tulajdonsagok, text="Tulajdonságok")

    #meghívjuk mindkét
    self.build_example_datas()
    self.build_empty_datas()
    self.show_rules()

    #Egy másodlagos frame, amibe a generált értékek kerülnek, viszont hibaüzenetkor törölhető
    self.secondary_frame = None
```

5.4. ábra. Részlet a *Matix_sum* – `create_screen()` metódusából.

Minden megoldás kérés amit a *SolutionController* megkap problémától függően, továbbítja a neki meghatározott kiszámító rendszeréhez, a solver pedig kiszámolja a kért eredményt, és megjeleníti arra az ablakra amire kérték.

5.3. Mátrix problémák bemutatása

Az alábbi fejezetekben részletesen ismertetem azokat a mátrix problémákat amelyeket az alkalmazásomban bemutattam, ezek tulajdonságait, képleteit, valamint hogyan oldottam meg egy számítógépes rendszeren belül. De mi is egy mátrix? A matematikában számok, szimbólumok, vagy kifejezések téglalap alakú elrendezése, amelyeket sorok és oszlopok

alkotnak. A benne tárolt értékeket elemeknek nevezzük, a mátrix vízszintes elrendezéseit soroknak, a függőlegeseket pedig oszlopoknak nevezzük. Egy mátrix $m \times n$ méretű, ha m sora és n oszlopa van és nagybetűkkel jelöljük, valamint a soraira és oszlopaira i és j -ként fogunk hivatkozni.

5.3.1. Mátrixok összeadása

Akkor értelmezzük két mátrix összeadását, amikor a típusaik (dimenziójuk) megegyeznek, és az összegmátrix i -dik sorának j -dik eleme az összeadandó mátrixok i -dik sorában és j -dik oszlopában levő elemek összege. Legyen egy A és B mátrix:

$$A = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \cdots & \alpha_{1j} \\ \alpha_{21} & \alpha_{22} & \cdots & \alpha_{2j} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{i1} & \alpha_{i2} & \cdots & \alpha_{ij} \end{bmatrix}, \quad (1)$$

$$B = \begin{bmatrix} \beta_{11} & \beta_{12} & \cdots & \beta_{1j} \\ \beta_{21} & \beta_{22} & \cdots & \beta_{2j} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{i1} & \beta_{i2} & \cdots & \beta_{ij} \end{bmatrix}, \quad (2)$$

akkor

$$A+B = A[\alpha_{ij}] + B[\beta_{ij}] = [\alpha_{ij} + \beta_{ij}]. \quad (3)$$

A mátrix feladatokat egy egyszerű *NumPy* függvénnyel is megoldhatjuk, viszont az egyszerűbb példánál a hagyományos ciklusos bejárást alkalmaztam, ahol sorról sorra, és oszlopról oszlopra bejártam az értékeket, majd pedig egyesével összeadtam ezeket és egy teljesen új összegmátrixba behelyeztem.

```
#két mátrix összeadása
# A[aij] + B[bij] = [aij + bij]
for i in range(len(matrix1_entries)):
    for j in range(len(matrix1_entries[i])):
        try:
            value1=int(matrix1_entries[i][j].get())
            value2=int(matrix2_entries[i][j].get())
            sum_matrix[i][j]=value1+value2
```

5.5. ábra. Részlet a *MatrixProblems* – *matrix_sum()* metódusából.

Miután az eredményt megkaptam egy egyszerű táblázatot generáltam ugyanabban a méretben mint az összegmátrix és feltöltöttem a megfelelő értékekkel.

5.3.2. Mátrixok szorzása

Két mátrix szorzata akkor létezik, ha a bal oldali A mátrix oszlopainak száma megegyezik a jobb oldali B mátrix sorainak a számával. A szorzásuk nem kommutatív vagy felcserélhető, még akkor sem, ha a mátrixok négyzetes mátrixok, tehát

$$A \cdot B \neq B \cdot A. \quad (4)$$

Meglepő mégis, hogy szorzatuk asszociatív, tehát csoportosítható, vagyis

$$(A \cdot B) \cdot C = A \cdot (B \cdot C). \quad (5)$$

Bár a mátrix szorzás nem-kommutatív, bármely mátrixhoz hozzárendelhető egy $n \times n$ -es E_n egységmátrix, ami semleges a szorzásra nézve, amelynek e_{ij} eleme 1, ha $i = j$, és ha $i \neq j$ akkor pedig 0, azaz

$$E_n = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}. \quad (6)$$

Akkor, ez könnyen ellenőrizhető,

$$E_m \cdot A = A \text{ és } A \cdot E_n = A, \quad (7)$$

Tehát a bal oldali E_m és jobb oldali E_n egységelem. Fontos megemlíteni azt is, hogy nem minden mátrix invertálható, ennek a tulajdonságait a megfelelő fejezetben részletezem.

A számítógépes környezetben, a mátrix szorzatánál a *NumPy* utat választottam, ahol a két mátrix-ot *NumPy* kompatibilis tömbbé alakítottam, majd pedig az erre szánt *np.matmul()* függvényt használva, kiszámoltam a két mátrix szorzatát.

```
#A két mátrix összeszorozása megfelelő numpy array formátumban
matrix1 = np.array(matrix1_values)
matrix2 = np.array(matrix2_values)
matrix_mult = np.matmul(matrix1,matrix2)
```

5.6. ábra. Részlet a *MatrixProblems* – *matrix_mult()* metódusából.

5.3.3. Transzponált mátrix

Egy n -edrendű $A = \alpha_{nn}$ mátrixot szimmetrikusnak mondunk, ha változatlan a sorainak és oszlopainak felcserélésére nézve, azaz ha $\alpha_{ij} = \alpha_{ji}$, minden i, j ($= 1, 2, \dots, n$) indexpár esetén.

Ha A^T -al jelöljük azt a mátrixot, amelyet úgy kapunk, hogy az $m \times n$ -es A mátrix sorait az oszlopaival felcseréljük, tehát, ha

$$A = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \cdots & \alpha_{1n} \\ \alpha_{21} & \alpha_{22} & \cdots & \alpha_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{m1} & \alpha_{m2} & \cdots & \alpha_{mn} \end{bmatrix}, \quad (8)$$

akkor

$$A^T = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \cdots & \alpha_{1n} \\ \alpha_{21} & \alpha_{22} & \cdots & \alpha_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{m1} & \alpha_{m2} & \cdots & \alpha_{mn} \end{bmatrix}. \quad (9)$$

Az így kapott A^T mátrixot az A *transzponáltjának* nevezzük. Egy A mátrix akkor szimmetrikus, ha megegyezik a transzponáltjával, tehát $A = A^T$.

Egy $A = \alpha_{nn}$ mátrix *adjungáltja* egy olyan $adj A$ -val jelölt $n \times n$ -es mátrix, mely annak a mátrixnak a transzponáltja, amit az A mátrix a_{ij} elemeinek $(-1)^{i+j} \det(A_{ij})$ -vel történő kicserélésével kaptunk, ahol A_{ij} az A mátrix a_{ij} eleméhez tartozó minormátrixa (az i, j eleme az A mátrix i -edik sorának és j -edik oszlopának törlésével keletkező mátrix determinánsa).

Az alkalmazásban a mátrix bekérése után minden elemet egyesével ellenőriztem, hogy rendes matematikai érték szerepeljen benne, majd pedig ezután a mátrixot transzponálható állapotba alakítottam, és transzponáltam.

```
#a mátrix numpy kompatibilis tömbbé alakítása
transposable=np.array(matrix_values)

#mátrix transzponálása
matrix_transposed=np.transpose(transposable)
```

5.7. ábra. Részlet a *MatrixProblems* – *matrix_transpose()* metódusából.

5.3.4. Egységmátrix

Az egységmátrix olyan $n \times n$ -es mátrix, ahol a főátlójában, tehát ahol az $i = j$ ott 1, és ahol $i \neq j$ -vel ott pedig 0. Az egységmátrixszal való szorzásnál az eredmény az a mátrix, ami az egységmátrixtól különbözik, tehát $A * E = A$. Az egységmátrix alakja:

$$E_n = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}. \quad (10)$$

Számítógépes környezetben roppant egyszerű létrehozni. Miután a felhasználó betáplálta, hogy az egységmátrix $n = ?$ értéke hány legyen, utána módosítva a véletlenszerűen értékfeltöltő rendszert, egy egyszerű dupla for ciklussal, valamint egy if ággal létrehoztam.

```
#egységmátrix értékekkel való feltöltése
def generate_data(self, entry_list):
    for i in range(len(entry_list)):
        for j in range(len(entry_list[i])):
            if(i==j):
                entry_list[i][j].insert(0, 1)
            else:
                entry_list[i][j].insert(0, 0)
```

5.8. ábra. Részlet a *matrixIdentity* – *generate_data()* metódusából.

5.3.5. Inverz mátrix

Egy mátrix invertálható, ha négyzetes mátrix, tehát $n \times n$ -es mátrix, és létezik szintén olyan négyzetes A^{-1} mátrix, amelyre

$$A \cdot A^{-1} = A^{-1} \cdot A = E_n = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}. \quad (11)$$

Ha $A \cdot A^{-1} = E_n$, akkor A^{-1} az a mátrix jobb oldali inverze, ha pedig $A^{-1} \cdot A = E_n$ akkor A^{-1} a mátrix bal oldali inverze. Egy mátrix invertálható, ha a determináns, tehát $\det A \neq 0$, akkor létezik kétoldali inverze. Valamint, ha A -nak létezik balinverze, vagy jobbinverze, akkor $\det A \neq 0$.

Ebből a két állításból az következik, hogyha egy mátrixnak létezik bal- vagy jobbinverze, akkor abból következik, hogy létezik a másik oldali inverz is, és bármelyik oldal létezése ekvivalens a $\det A \neq 0$ feltétellel.

Az inverz mátrixot a következő képlettel kapjuk:

$$A^{-1} = \frac{1}{\det A} \cdot \text{adj } A. \quad (12)$$

Egy számítógépes környezetben a *NumPy* *np.linalg.inv()* függvényt használva valósítottam meg, ami invertálja a mátrixot, ha a $\det A \neq 0$.

```
#előkészítés az invertálásra
invertable = np.array(matrix_values)

#mátrix invertálása
#hiba esetén exception-t dob, és hibaüzenetet ír ki (determináns 0)
if (np.linalg.det(invertable)!=0):
    try:
        result=np.linalg.inv(invertable)
```

5.9. ábra. Részlet a *MatrixProblems* – *matrix_invert()* metódusából.

5.3.6. Mátrix determinánsa

„Az A mátrix determinánsa alatt egy olyan T -beli elemet értünk, amelyet a következőképpen határozunk meg. Először is n -tényezős szorzatokat képezünk minden lehetséges módon úgy, hogy a mátrix minden sorából és minden oszlopából pontosan egy tényezőt veszünk. A következő lépésben minden egyes szorzatot a “+” vagy “-” előjellel látunk el. Ez azt jelenti, hogy vagy magát a szorzatot, vagy pedig a negatívját tekintjük. Végül ezeket az előjeles szorzatokat összeadjuk. Az így kapott összeget nevezzük az A mátrix determinánsának.” (Fraud, 1996)

$$\det A = \begin{vmatrix} \alpha_{11} & \alpha_{12} & \dots & \alpha_{1n} \\ \alpha_{21} & \alpha_{22} & \dots & \alpha_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{n1} & \alpha_{n2} & \dots & \alpha_{nn} \end{vmatrix} = \sum_{\sigma} (-1)^{I(\sigma)} \alpha_{1\sigma(1)} \alpha_{2\sigma(2)} \dots \alpha_{n\sigma(n)}. \quad (13)$$

A jobb oldalon látható $\sum$ összegzést az $1, 2, \dots, n$ számok minden lehetséges σ permutációjára el kell végezni. Az összegben az $n!$ tagnak pontosan fele van ellátva negatív előjelzéssel, hiszen ugyanannyi páratlan és páros permutáció van.

Számítógépes környezetben, az invertálhatóságnál már látott (5.9. ábra) `np.linalg.det()` függvényt alkalmazva számoltam ki a determinánst, ami a *NumPy* egyik függvénye.

```
#mátrix determináns számításánál előkészítése
matrix = np.array(matrix_values)
try:
    #eredmény kerekítése kettes tizedes jegyig
    result=round(np.linalg.det(matrix),2)
```

5.10. ábra. Részlet a *MatrixProblems* – `matrix_determinant()` metódusából.

Az eredményt kiszámolva ezután egy *Tk.Label* -t alkalmazva egyszerűen kiíratunk az ablakra.

5.3.7. Háromszögmátrix

A háromszögmátrix egy olyan $n \times n$ -es négyzet mátrix, amelynek a főátlója egyik oldalán csak 0-sok szerepelnek. Ez lehet kétféle háromszögmátrix, attól függően, hogy a 0-sok a főátló mely oldalán szerepelnek. Lehet alsó háromszögmátrix, ahol az átló felett helyezkednek el, valamint lehet felső háromszögmátrix is, ahol a főátló alatt helyezkednek el a 0-sok.

- Felső háromszögmátrix alakja:

$$A = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \cdots & \alpha_{1n} \\ 0 & \alpha_{22} & \cdots & \alpha_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \alpha_{nn} \end{bmatrix}. \quad (14)$$

- Alsó háromszögmátrix alakja:

$$A = \begin{bmatrix} \alpha_{11} & 0 & \cdots & 0 \\ \alpha_{21} & \alpha_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{n1} & \alpha_{n2} & \cdots & \alpha_{nn} \end{bmatrix}. \quad (15)$$

Fontos megemlíteni, hogy a háromszögmátrix determinánsa roppant egyszerűen kiszámítható, mivel egyszerűen csak össze kell szorozni a főátlóban lévő elemeket:

$$\det A = \alpha_{11} \cdot \alpha_{22} \cdot \dots \cdot \alpha_{nn}. \quad (16)$$

Fontos azt is megemlíteni, hogy egy háromszögmátrix csakis akkor invertálható, ha a főátlójában egyetlen 0 sem szerepel. Valamint két felső vagy két alsó háromszögmátrix összege vagy szorzata szintén felső, vagy alsó háromszögmátrix marad, attól függően melyikkel végeztünk műveletet.

Egy digitális környezetben, egy ilyen háromszögmátrix típusának az eldöntését egyszerűen el lehet végezni egy *Tkinter* Rádiogomb használatával, ahol megadjuk a lehetőséget a felhasználónak, hogy eldöntse, hogy alsó vagy felső háromszögmátrixot szeretne. Természetesen egyszerre csak az egyik opciót választhatja ki. A mátrix generálását, és feltöltését a fentiekben megismert *generate_data()* (5.8. ábra) egy módosított verzióját használtam, ahol úgy alakítottam az értékek generálását, hogy az előbb említett két opció közül valamelyiknek megfelelően az értékfeltöltés.

```
#háromszög mátrix generálása
def generate_data(self, entry_list):
    for i in range(len(entry_list)):
        for j in range(len(entry_list[i])):

            #ha felső háromszögmátrix
            if(int(self.selected_triangle.get())==1):
                if (i == j):
                    entry_list[i][j].insert(0, random.randint(a: 0, b: 50))

                elif(i<j):
                    entry_list[i][j].insert(0, random.randint(a: 0, b: 50))
                else:
                    entry_list[i][j].insert(0, 0)

            #ha alsó háromszögmátrix
            else:
                if (i == j):
                    entry_list[i][j].insert(0, random.randint(a: 0, b: 50))
                elif(i > j):
                    entry_list[i][j].insert(0, random.randint(a: 0, b: 50))
                else:
                    entry_list[i][j].insert(0, 0)
```

5.11. ábra. Részlet a *matrixTriangle* – *generate_data()* metódusából.

5.3.8. Cramer-szabály

Az alábbi fejezetben olyan speciális egyenletrendszerekről lesz szó, amelyekben az ismeretlenek száma megegyezik az egyenletek számával. Az együtthatómátrix négyzetes, ezért létezik determinánsa.

Ha A mátrix egy $n \times n$ -es mátrix, és $D = \det A \neq 0$, akkor az $A \cdot x = b$ egyenletrendszernek pontosan 1 megoldása van. A megoldásban $x_j = \frac{D_j}{D}$, ahol a D_j determinánst úgy kapjuk, hogy D -ben a j -edik oszlop helyére a jobb oldali konstansokat (szabadtagokat), azaz a b vektor komponenseit írjuk. Van az alábbi A mátrix, és b vektor:

$$A = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \dots & \alpha_{1n} \\ \alpha_{21} & \alpha_{22} & \dots & \alpha_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{n1} & \alpha_{n2} & \dots & \alpha_{nn} \end{bmatrix}, \quad (17)$$

$$b = \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_n \end{bmatrix}. \quad (18)$$

A fenti $x_j = \frac{D_j}{D}$ képletet alkalmazva kiszámoljuk az ismeretleneket, például ha a második ismeretlent szeretnénk kiszámolni akkor:

$$x_2 = \frac{\begin{vmatrix} \alpha_{11} & \beta_1 & \dots & \alpha_{1n} \\ \alpha_{21} & \beta_2 & \dots & \alpha_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{n1} & \beta_n & \dots & \alpha_{nn} \end{vmatrix}}{\begin{vmatrix} \alpha_{11} & \alpha_{12} & \dots & \alpha_{1n} \\ \alpha_{21} & \alpha_{22} & \dots & \alpha_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{n1} & \alpha_{n2} & \dots & \alpha_{nn} \end{vmatrix}}. \quad (19)$$

Mivel a feltételek szerint létezik A^{-1} , ezért az $A \cdot x = b$ egyenletrendszert balról A^{-1} -gyel beszorozva ekvivalens egyenletrendszert nyerünk. Így az $x = A^{-1} \cdot b$ egyenletrendszer keletkezett, ami már „meg is van oldva”. Ezzel igazolni is tudtuk, hogy az eredeti egyenletrendszernek pontosan egy megoldása van.

A mátrix szorzás szabályai szerint x_j , az A^{-1} mátrix j -dik sorának, és a b vektornak a szorzata. Felhasználva a mátrix inverz képletét: $x_j = \frac{1}{D} \cdot (\alpha_{1j}\beta_1 + \dots + \alpha_{nj}\beta_n)$.

Mivel a D_j determináns kizárólag a D determináns j -dik oszlopában tér el a D -től, ezért a j -dik oszlophoz tartozó megfelelő előjeles aldeterminánsok D -ben és D_j -ben azonosak. Ebből

következik, hogy D_j -t a j -dik oszlopa szerint kifejezve $D_j = \alpha_{1j}\beta_1 + \dots + \alpha_{nj}\beta_n$ adódik, tehát x_j valójában $x_j = \frac{D_j}{D}$.

Fontos megemlíteni, hogy a Cramer-szabálynak elsősorban elméleti jelentősége van, hiszen az egyenletrendszerek megoldásában ritkán használjuk csak, mivel egyrészt eleve csak speciális esetekben alkalmazható, másrészt sokkal több számolást igényel, mint a Gauss-kiküszöbölés.

Számítógépes rendszerekben persze egyszerű erre egy dinamikus rendszert építeni ami Cramer-szabály segítségével számolja ki a megoldást, hiszen felhasználva egy processzor rendkívülien gyors számítási sebességét, egy időigényes számítást is pillanatokon belül ki tudunk számolni, és kódtól függően, akár egy rekurzív függvény, vagy ciklus segítségével kiszámítani ezeket.

Az alkalmazásban írtam egy egyszerű *for* ciklust, ami annyiszor fut le, ahány ismeretlenünk van, mindig kicserélve az adott oszlopot az eredményvektorral, kiszámolva az adott ismeretlent, ezt pedig a felhasználónak megjeleníti.

```
#oszloponként végigmegyünk az egységmátrixon és kicseréljük a vektorra
#matrix_temp => kicserélt mátrix
for column in range(len(matrix1_values)):
    matrix_temp=[None] * len(row) for row in matrix1_entries
    for i in range(len(matrix1_values)):
        for j in range(len(matrix1_values[i])):
            if(column==j):
                matrix_temp[i][j]=matrix2_values[0][i]
            else:
                matrix_temp[i][j] = matrix1_values[i][j]

#az ideiglenes mátrix felkészítése determináns számolásra
temporary=np.array(matrix_temp)

#ismeretlen számolás az xj = Dj/D képlet alapján
result_cramer=round(np.linalg.det(temporary)/matrix_det,2)

#ismeretlenek ablakba való kiírása, a,b,c... nevekként
ttk.Label(self.results, text=chr(97+column)+" = " + str(result_cramer)).grid(row=0+column, column=0)
self.results.grid(row=0, column=0)
tab.grid(row=5, column=0, columnspan=2)
```

5.11. ábra. Részlet a *matrixProblems* – *matrix_cramer()* metódusából.

5.4. Vektorproblémák bemutatása

Az ezt követő fejezetekben részletesen fogok beszélni a vektorproblémákról, és ezekből fogok bemutatni pár példát, valamint tulajdonságaikat, képleteit, és ahogyan egy számítógépes környezetben prezentálhatóak. Egy irányított szakaszt nevezünk vektornak, ezeket jelölhetjük: $\mathbf{u}, \mathbf{v}, \mathbf{x}, \vec{i}, \vec{j}, \vec{k}, \overline{AB}, \overline{AB}$. Egy vektort három dolog jellemez: irány, irányítás, és hossz.

A vektorokat eltolhatjuk, ha két vektor iránya, irányítása, és hossza megegyezik akkor eltolással a vektorok pontosan lefedik egymást. Ebben az esetben ekvivalensnek $\bar{u} \equiv \bar{v}$ vagy egyenlőnek nevezzük $\bar{u} = \bar{v}$. (Kupán, 2019)

Számítógépes környezetben szinte mindenhol háromdimenziós térben kerülnek bemutatásra, hiszen a leglátványosabb, ha a felhasználó ezt a teret körbe tudja forgatni és alaposan megvizsgálni az adott vektort, vektorokat.

5.4.1. Vektorok normája

Egy $\bar{v}$ vektor hosszát *normának* nevezzük és $\|\bar{v}\|$ -vel jelölünk. Az $\mathbb{R}^2$ síkban a $\bar{v} = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix}$ vektor normáját a Pitagorasz tétellel számoljuk ki:

$$\|\bar{v}\| = \sqrt{v_1^2 + v_2^2}. \quad (20)$$

Hasonlóan járunk el $\mathbb{R}^3$ -ban:

$$\|\bar{v}\| = \sqrt{v_1^2 + v_2^2 + v_3^2}, \quad (21)$$

$$\text{ahol } \bar{v} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} \in \mathbb{R}^3. \quad (22)$$

Számítógépes környezetben a képlet alkalmazása roppant egyszerű, a *NumPy* rendelkezik egy *np.linalg.norm()* függvénnyel, képes kiszámolni két- és háromdimenziós vektorokat is, amely felgyorsítja a normaszámítást, viszont ehhez a képlethez a hagyományos megközelítést végeztem.

```
#norma kiszámítása pitagorasz tétel segítségével
vector_norm = ( vector_matrix[0]**2 + vector_matrix[1]**2 + vector_matrix[2]**2 )**(1/2)
```

5.12. ábra. Részlet a *vectorProblems* – *vector_norm()* metódusából.

Mindezek mellett egy vektor kirajzolása nem egy nehéz folyamat, pár egyszerű egymást követő lépést kell elvégezni, ezek nagyon hasonlóak mindegyik később prezentált problémánál, a nagyobb különbség inkább a vektorok száma, valamint az, ahogy ezek a vektorok hogyan kerülnek kiszámításra.

Ahhoz hogy elkezdjük felépíteni az adott 2D, vagy 3D-s diagrammot amibe a vektorunkat megrajzoljuk, pár nagyon egyszerű lépést kell elvégezzünk. El kell döntenünk hány dimenziós diagrammot használunk, felállítani a határt a diagramnak, ez lényegében azért fontos, hogy mennyire legyen úgymond ráközelítve a diagramra. Ezután kirajzoljuk a vektort azon paraméterek szerint ahogy kívánjuk, felcímkézzük a diagrammot, és kirajzoljuk. Ezen említett lépések megvalósítása az alábbi 5.13. ábrán látható.

```

# 3D diagram előkészítése, méretezése
fig = plt.figure(figsize=(4, 4))
ax = fig.add_subplot(111, projection="3d")

# diagram határának meghatározása
lim = max(1.0, np.abs(vector_matrix).max() * 1.2)
min_lim=min(0, np.abs(vector_matrix).min())
ax.set_xlim(min_lim, lim)
ax.set_ylim(min_lim, lim)
ax.set_zlim(min_lim, lim)

# vektor megrajzolása
ax.quiver(
    *vector_origo,
    *vector_matrix,
    color='red', linewidth=3,
    arrow_length_ratio=0.15, label=f"A vektor hossza (norma): {round(vector_norm, 2)}"
)

# canvas amit az ablakra vetítünk
canvas = FigureCanvasTkAgg(fig, master=self.results)

# diagram felcímkézése
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Vektor Hossza')
ax.legend()

# diagram kirajzolása az ablakba
canvas.draw()
canvas.get_tk_widget().grid(row=1, column=0)

```

5.13. ábra. Részlet a `vectorProblems – vector_norm()` metódus diagram rajzolásáról.

5.4.2. Vektorok összeadása

Két vektor összeadása kommutatív és asszociatív művelet:

$$\vec{u} + \vec{v} = \vec{v} + \vec{u}, \quad (23)$$

$$(\vec{u} + \vec{v}) + \vec{w} = \vec{v} + (\vec{u} + \vec{w}). \quad (24)$$

Egy zero hosszúságú vektort *nullvektornak* nevezünk, és 0-val vagy θ -vel jelöljük:

$$\vec{v} + \theta = \vec{v}. \quad (25)$$

Egy $\vec{v}$ -vel azonos irányú, és hosszúságú de ellentétes irányítású vektort ($-\vec{v}$) *ellentett vektornak* nevezzük:

$$\vec{v} + (-\vec{v}) = \theta. \quad (26)$$

Két vektor összeadásakor minden koordinátát a saját koordinátájához adjuk hozzá a következőképpen:

$$\vec{v} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} + \vec{u} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \vec{w} \begin{pmatrix} v_1 + u_1 \\ v_2 + u_2 \\ v_3 + u_3 \end{pmatrix}. \quad (27)$$

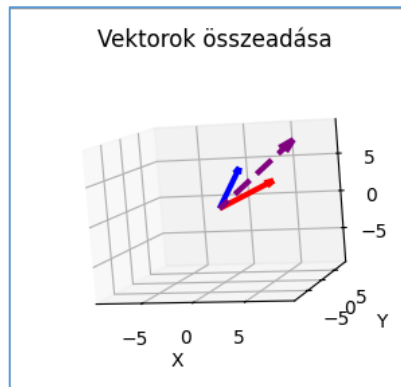
Ezt egy egyszerű ciklussal, a két vektort pillanatok alatt össze lehet adni, és a harmadik vektort kiszámítva, majd pedig kirajzolva kiválóan látható, hogy a két vektor összeadása, és közös kiinduló pont miatt, a harmadik vektor a két vektor közepén fog elhelyezkedni.

```

# két vektor összeadása
1 usage
def vector_sum(self, diagram, vector1, vector2):
    # összeadott vektor előkészítése
    sum_vector = []
    # ciklus amivel a két vektort összeadom
    for i in range(len(vector1)):
        sum_vector.append(vector1[i] + vector2[i])

```

5.14. ábra. Részlet a `vectorProblems – vector_sum()` metódusából.



5.15. ábra. Két vektor összeadása diagrammon

5.4.3. Skalárral való szorzás

A skalárral való szorzás alatt azt értjük, hogy egy vektor minden elemét beszorozzuk egy számmal, ezt a számot skalárnak nevezzük, és görög betűvel jelöljük, például λ skalár. Az eredmény pedig egy újonnan létrehozott vektor, amelynek az értékei pontosan az előző vektor értékei beszorozva a skalárral.

A skalárral való szorzás disztributív, tehát szétosztható mind a vektorösszeadásra, mind a skalárok összeadására nézve. Például ha két skalárt összeadunk, és azzal szorzunk meg egy vektort, az megegyezik azzal, mintha a vektort külön-külön megszoroznánk a skalárokkal, majd a kapott vektorokat összeadnánk:

$$(\lambda + \mu)\vec{v} = \lambda\vec{v} + \mu\vec{v}, \quad (28)$$

A skalárral való szorzás asszociatív is a skalárok szorzására nézve, vagyis ha két skalárral is meg szeretnénk szorozni egy vektort, akkor teljesen mindegy melyik sorrendben szorozzuk be, hiszen ugyanazt az eredményt kapjuk:

$$(\gamma\mu)\vec{v} = \gamma(\mu\vec{v}) \quad (29)$$

Egy vektort ha beszorzol 1-el nem változtatja meg a vektort, ez bizonyítja, hogy a skalárral való szorzás normálisan működik, és nem fordul el, vagy torzul el a vektor:

$$1 \cdot \vec{v} = \vec{v}. \quad (30)$$

Viszont fontos megemlíteni, hogy a (-1) -el való szorzás teljesen megfordítja a vektor irányítását, és az ellenkező irányba fog mutatni. Egy skalárral való szorzás általános alakja, ahol a $\vec{v}$ vektort beszorzunk egy λ skalárral, ezzel kiszámolva egy új $\vec{w}$ vektort:

$$\vec{v} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} \cdot \lambda = \vec{w} \begin{pmatrix} v_1 \cdot \lambda \\ v_2 \cdot \lambda \\ v_3 \cdot \lambda \end{pmatrix}. \quad (31)$$

Mint két vektor összeadásánál, itt is úgy járunk el, és szorozzuk össze a skalárt a vektor értékeivel, tehát egy ciklus alkalmazásával, ami segítségével roppant egyszerűen létrehozuk az új vektort, és máris kirajzolhatjuk a diagrammra.

```
#Skalárral való szorzás
1 usage
def vector_mult(self, diagram, vector, scalar):

    #új vektor előkészítése
    mult_vector = []

    #skalárral való szorzás elemről elemre
    for i in range(len(vector)):
        mult_vector.append(vector[i] * scalar)
```

5.16. ábra. Részlet a `vectorProblems` – `vector_mult()` metódusából.

5.4.4. Lineáris kombináció

A lineáris kombináció a lineáris algebra egyik legfontosabb fogalma, hiszen segítségével definiálható a mátrixok rangja, a vektorrendszerek, valamint a vektorok lineáris függetlensége is. Két vektor, $\vec{u}$ és $\vec{v}$ lineáris kombinációján azt a műveletet értjük, amikor a vektorokat megszorozzuk egy-egy skalár számmal (λ és μ), majd az így kapott szorzatvektorokat összeadjuk:

$$\lambda \vec{u} + \mu \vec{v} = \vec{w}. \quad (32)$$

Mivel a vektorokat bizonyos skalárral össze kell szorozni, ezután a szorzatokat összeadni, ezért a lineáris kombináció vektortéren értelmezhető. Veszünk pár vektort, ekkor az összes lineáris kombinációjuk szintén vektorteret ad, valamint ezek a vektorok által létrehozott vektorteret más néven *lineáris buroknak* hívjuk.

Hasonlóan kerül kiszámításra egy digitális környezetben mint az fenn említett skalárral való szorzás és vektor összeadás, ezt a kettőt egybefonva kiszámoljuk a harmadik vektort, és kirajzoljuk a diagramra ugyanúgy.

```

# lineáris kombináció kiszámítása
1 usage
def vector_linear_combination(self, diagram, vector1, vector2, scalars):
    mult_vector_1 = []
    mult_vector_2 = []

    # a két vektor skaláris szorzatának kiszámítása
    for i in range(len(vector1)):
        mult_vector_1.append(vector1[i] * scalars[0])

    for i in range(len(vector2)):
        mult_vector_2.append(vector2[i] * scalars[1])

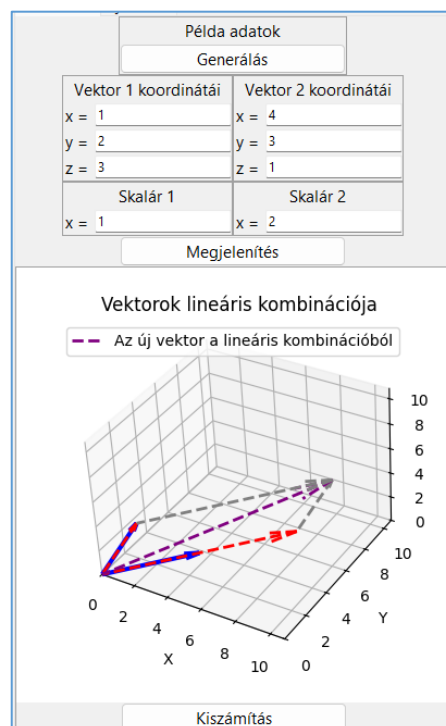
    vector_combination = []

    # az új kiszámolt vektor lineáris kombináció segítségével
    for i in range(len(mult_vector_1)):
        vector_combination.append(mult_vector_1[i] + mult_vector_2[i])

```

5.17. ábra. Részlet a `vectorProblems – vector_linear_combination()` metódusából.

Az alábbi ábrán is kiválóan látható, miként jelenik meg, és hogyan viselkedik a lineáris kombináció.



5.18. ábra. Részlet a `Vektorok lineáris kombinációja` ablakról

5.4.5. Két vektor skaláris szorzata

Két vektor skaláris szorzata alatt az

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \|\vec{v}\| \cos \varphi \quad (33)$$

számot értjük, ahol $\varphi \in [0, \pi)$ a két vektor által bezárt szög, $\|\vec{u}\|$ és $\|\vec{v}\|$ pedig a két vektor hossza, vagyis a normája.

Két vektor merőleges egymásra, akkor és csakis akkor, ha a két vektor skaláris szorzata zéró. Azaz:

$$\vec{u} \perp \vec{v} \Leftrightarrow \varphi = \frac{\pi}{2} \Leftrightarrow \cos \varphi = 0 \Leftrightarrow \vec{u} \cdot \vec{v} = 0. \quad (34)$$

A két vektor által bezárt szöget tehát ki tudjuk számolni az alábbi képlet segítségével:

$$\cos \varphi = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}. \quad (35)$$

A két vektor skaláris szorzatát egyszerűen úgy is ki tudjuk számolni, hogy a két vektor elemeit összeszorozzuk egymással, majd pedig ezeket összeadjuk, tehát:

$$\vec{u} \cdot \vec{v} = u_1 v_1 + u_2 v_2 + u_3 v_3 \quad (36)$$

A számítógépes térben ugyanígy járunk el, hogy a két vektor elemeit egyesével összeszorozzuk, és összeadjuk, majd a kapott érték segítségével és a *NumPy* már bemutatott *np.linalg.norm()* vektor norma kiszámító függvényével alkalmazzuk a $\cos \varphi$ kiszámító képletet. Miután kiszámoltuk, alkalmazzuk az *np.arccos()* függvényt a már kiszámolt $\cos \varphi$ -ra, hogy megkapjuk a pontos szöget, valamint alkalmazzuk a *np.degrees()*-t, ami a már kiszámított szöget fokokká alakítja, amit már egyszerűen ki tudunk írni a felhasználónak.

```
# a két vektor skaláris szorzat kiszámítása
1 usage
def vector_scalar_mult(self, tab, vector1, vector2):
    # skaláris szorzat kiszámítása
    # értékek szorzatának összege
    scalar_value = 0
    for i in range(len(vector1)):
        scalar_value += vector1[i] * vector2[i]

    # két vektor közötti szög koszinusz kiszámítása
    cos_a = scalar_value / (np.linalg.norm(vector1) * np.linalg.norm(vector2))

    # előző eredmény törlése, majd felkészítése egy új eredményre
    if self.results is not None:
        self.results.destroy()

    self.results = ttk.Frame(tab)

    # skaláris szorzat kiírása
    ttk.Label(self.results, text="2 vektor skaláris szorzata : " + str(round(scalar_value, 2))).grid(row=5, column=0)

    # a vektorok által bezárt szög pontos foka
    ttk.Label(self.results, text=f"u és v által bezárt szög : {round(np.degrees(np.arccos(cos_a)), 2)}°").grid(row=6, column=0)
    self.results.grid(row=7, column=0)
```

5.19. ábra. Részlet a *vectorProblems* – *vector_scalar_mult()* metódusából.

5.4.6. Vektorok vektoriális szorzata

Két vektor $\bar{u}, \bar{v} \in \mathbb{R}^3$ vektoriális szorzatán azt a $\bar{z} = \bar{u} \times \bar{v}$ vektort értjük, amelynek az iránya merőleges az $\bar{u}$ és $\bar{v}$ vektorok síkjára, tehát:

$$\bar{z} \perp (\bar{u}, \bar{v}). \quad (37)$$

Továbbá az irányítása a jobb kéz szabály szerint történik, valamint a hossza:

$$\|\bar{z}\| = \|\bar{u}\| \|\bar{v}\| \sin \varphi, \quad (38)$$

ahol $\varphi \in [0, \pi)$ a két vektor által bezárt szög.

A $\bar{z}$ vektor irányításából következik, hogy $\bar{u} \times \bar{v} = -(\bar{v} \times \bar{u})$. Továbbá két vektor akkor és csak akkor párhuzamos egymással, hogyha a vektoriális szorzatuk egyenlő a nullvektorral.

A fenti összefüggéseket figyelembe véve kiszámíthatjuk az

$$\bar{u} = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = u_1 \bar{i} + u_2 \bar{j} + u_3 \bar{k} \text{ és } \bar{v} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} = v_1 \bar{i} + v_2 \bar{j} + v_3 \bar{k} \quad (39)$$

vektorok vektoriális szorzatát:

$$\begin{aligned} \bar{u} \times \bar{v} &= (u_1 \bar{i} + u_2 \bar{j} + u_3 \bar{k}) \times (v_1 \bar{i} + v_2 \bar{j} + v_3 \bar{k}) = \\ &= u_1 v_1 \bar{i} \times \bar{i} + u_1 v_2 \bar{i} \times \bar{j} + \dots + u_3 v_3 \bar{k} \times \bar{k} = u_1 v_2 \bar{k} - u_1 v_3 \bar{j} + \dots + u_3 v_2 \bar{i}, \end{aligned} \quad (40)$$

ami determináns segítségével felírható az alábbi alakra:

$$\bar{z} = \bar{u} \times \bar{v} = \begin{vmatrix} \bar{i} & \bar{j} & \bar{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix}. \quad (41)$$

A három koordináta az új vektorhoz kiszámítható az alábbi három képlet segítségével:

$$z_1 = u_2 \cdot v_3 - u_3 \cdot v_2, \quad (42)$$

$$z_2 = v_1 \cdot u_3 - u_1 \cdot v_3, \quad (43)$$

$$z_3 = u_1 \cdot v_2 - u_2 \cdot v_1. \quad (44)$$

A fentiek ismerete alapján egyszerűen létre tudtam hozni a rendszert, amivel a két vektort vektoriálisan összeszoroztam, miután a felhasználó betáplálta a kívánt vektorait. A megfigyelések alapján a program helyesen működik, a kiszámolt vektor mindig merőleges a másik két vektorra.

```

# vektoriális szorzat kiszámítása
1 usage
def vectorial_mult(self, tab, diagram, vector1, vector2):

    # kapott vektor előkészítése
    vectorial_mult = []

    # x coord
    coord = (vector1[1] * vector2[2]) - (vector1[2] * vector2[1])
    vectorial_mult.append(coord)

    # y coord
    coord = (vector1[2] * vector2[0]) - (vector1[0] * vector2[2])
    vectorial_mult.append(coord)

    # z coord
    coord = (vector1[0] * vector2[1]) - (vector1[1] * vector2[0])
    vectorial_mult.append(coord)

    # kapott vektor
    vectorial_vector = np.array(vectorial_mult)

    vectorial_vector_norm = np.linalg.norm(vectorial_vector)
    norm_1 = np.linalg.norm(vector1)
    norm_2 = np.linalg.norm(vector2)
    sin_a = vectorial_vector_norm / (norm_1 * norm_2)

    # u és v vektor hajlásszöge
    radian = np.arcsin(sin_a)
    degree = np.degrees(radian)

```

5.20. ábra. Részlet a *vectorProblems* – *vectorial_mult()* metódusából.

5.4.7. Háromszög megoldás

Bármely 3 vektor alkalmazásával egy tetszőleges háromszöget rajzolhatunk a térben, és vektorok segítségével a háromszög tulajdonságait képesek vagyunk meghatározni. Képesek vagyunk kiszámolni a háromszög területét, kerületét, a három szög mértékét, a háromszög köré írt kör sugarát, a háromszögbe írt kör sugarát, valamint bármelyik csúcshoz tartozó magasság hosszát is.

A háromszög kerülete, ami a három oldal összege, egyszerűen kiszámolható a három megadott vektorunk hosszának összegeként. Területnél alkalmazzuk az előbb említett vektoriális szorzást, hiszen az ebből adódó paralelogramma területének a fele pontosan a háromszögünk területével lesz egyenlő.

Kiszámíthatjuk a háromszög súlypontját is, amit úgy számolunk ki, hogy a három pont koordinátáit összeadjuk, majd pedig elosztjuk hárommal, tehát:

$$G = \frac{A + B + C}{3}. \quad (45)$$

A háromszög szögeit a vektorok skaláris szorzatánál említett módszerrel és képlettel számoltam ki, ezt megtettem mind a három szög kiszámításához. Miután az alap paramétereket, tehát a kerület, terület, normákat kiszámoltuk, utána egyszerűen kiszámíthatóvá válik a háromszög köré írt kör sugara is, amit úgy számolunk ki, hogy lényegében összeszorozzuk a három vektor hosszát, majd pedig elosztuk a terület négyszeresével.

$$\Delta \text{ köré írt kör sugara} = \frac{A_{norm} \cdot B_{norm} \cdot C_{norm}}{4 \cdot \text{Terület}} \quad (46)$$

A háromszögbe írt kör sugarát kiszámoltam az alábbi képlet segítségével:

$$\Delta \text{ -be írt kör sugara} = \frac{2 \cdot \text{terület}}{\text{kerület}}. \quad (47)$$

Az A csúcshoz tartó magasság kiszámításához egyszerűen alkalmaztam a következő képletet:

$$A \text{ - hoz tartó magasság} = \frac{2 \cdot \text{terület}}{\overrightarrow{BC}_{norm}} \quad (48)$$

Ahogy az megfigyelhető, három vektor megadásából meglepően sok tulajdonságot megvizsgálhatunk, kiszámíthatunk és bemutathatunk egy háromszöggel kapcsolatban. Egy 3D-s környezetben ezek a tulajdonságok még szebben és jobban láthatóak, főleg amikor ez a diagramm interaktív, és a felhasználó kedve szerint mozgathatja, és forgathatja a háromszöget a térben.

```
#kerulet szamitas
perimeter = 0
perimeter += np.linalg.norm(ab_vector)
perimeter += np.linalg.norm(bc_vector)
perimeter += np.linalg.norm(ca_vector)

ttk.Label(self.results, text="A háromszög kerülete : " + str(round(perimeter, 2))).grid(row=5, column=0)

#terulet szamitas
vectorial_mult = []

coord = (ab_vector[1] * bc_vector[2]) - (ab_vector[2] * bc_vector[1])
vectorial_mult.append(coord)

coord = (ab_vector[2] * bc_vector[0]) - (ab_vector[0] * bc_vector[2])
vectorial_mult.append(coord)

coord = (ab_vector[0] * bc_vector[1]) - (ab_vector[1] * bc_vector[0])
vectorial_mult.append(coord)

vectorial_vector = np.array(vectorial_mult)

area = round(np.linalg.norm(vectorial_vector) / 2, 2)

ttk.Label(self.results, text="A háromszög területe : " + str(round(area, 2))).grid(row=6, column=0)
self.results.grid(row=7, column=0)
```

5.21. ábra. Részlet a `vectorProblems – triangle_solution()` metódusából.

5.5. Vektorterek bemutatása

„Legyen K egy kommutatív test (leggyakrabban R, C, Z_2). A $V \neq 0$, halmazon értelmezünk egy belső (összeadás $+$) és egy külső (skalárral való szorzás) műveletet: ” (Kupán, 2019)

- belső művelet : $V \times V \rightarrow V, \forall u, v \in V \Rightarrow \exists! u + v \in V$;
- külső művelet : $K \times V \rightarrow V, \forall \lambda \in K, v \in V \Rightarrow \exists! \lambda \cdot v \in V$.

A V halmazt K feletti vektortérnek nevezzük ha teljesülnek az alábbi összeadásra, és skalárral való szorzásra vonatkozó axiómák:

1. Bármely két $\bar{u}$ és $\bar{v}$ vektor összege szintén a V halmazban van, tehát létezik zártság az összeadásra.
2. kommutativitás: $\bar{u} + \bar{v} = \bar{v} + \bar{u}$, minden $\bar{u}, \bar{v}$ eleme V -re.
3. Asszociativitás: $(\bar{u} + \bar{v}) + \bar{w} = \bar{u} + (\bar{v} + \bar{w})$.
4. Létezik nullvektor: olyan 0 vektor V -ben, amelyre $\bar{u} + 0 = \bar{u}$ teljesül minden $\bar{u}$ -ra.
5. Létezik ellentett vektor: Minden V -beli u vektorhoz létezik olyan $-\bar{u}$ vektor V -ben, amelyre $\bar{u} + (-\bar{u}) = 0$.
6. Bármely u vektor c skalárral vett szorzata szintén a V halmazban van.
7. Disztributivitas vektorösszegre: $c(\bar{u} + \bar{v}) = c\bar{u} + c\bar{v}$.
8. Disztributivitas skalárösszegre : $(c + d)\bar{u} = c\bar{u} + d\bar{u}$.
9. Skalárszorzat asszociativitása: $c(d\bar{u}) = (cd)\bar{u}$.
10. Egységelem tulajdonsága: $1\bar{u} = \bar{u}$, ahol 1 a skalártest egységeleme.

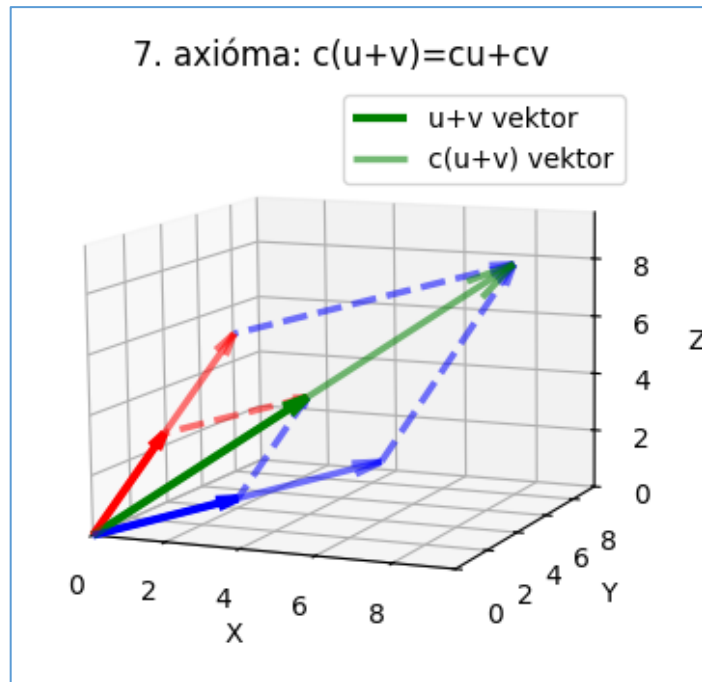
Valós vektortér ahol összeadás illetve szorzás műveleteket komponensenként végzünk:

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} x_1 + y_1 \\ x_2 + y_2 \\ x_3 + y_3 \end{pmatrix}, \alpha \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} \alpha x_1 \\ \alpha x_2 \\ \alpha x_3 \end{pmatrix}. \quad (49)$$

Ha $K = R$ akkor valós vektortérrel, ha pedig $K = C$ akkor komplex vektortérrel beszélünk.

Az alkalmazásomban a hetedik axiómát mutatom be, a disztributivitást vektorösszegre, ahol kiválóan látható az axióma viselkedése. Először elvégeztem az egyenlet bal oldalát, tehát kiszámoltam a két vektor összegét, ez a vastag zöld vektor, majd pedig kiszámoltam a vektornak a skalárral való szorzatát, ami a halványzöld vektor.

Miután a bal oldalt elvégeztem, kiszámoltam az egyenlet jobb oldalát, ahol az alap, sötétpirossal és kékkel jelölt vektorokat megszoroztam a skalárral külön-külön, ezek a halvány folytonos vonallal megrajzolt vektorok, majd ezeket összeadtam. Ahogyan az ábrán is látható az adott axióma kiválóan bemutatja a disztributivitást a vektorösszegre.



5.22. ábra. 7. axióma bemutatása.

Ezt a vektoroknál már bemutatott egyszerű skalárral való szorzással és vektorok összeadásával értem el, valamint a segédvektorokat egyszerű kivonással kiszámoltam, hogy ahova szeretném hogy tartson, kivontam abból azt a vektort, ahonnan szeretném hogy induljon, majd pedig kiindulópontnak szintűgy ezt a vektort használtam, ahonnan szerettem volna, hogy induljon. Ezek a segédvektorok kiválóan mutatják az arányosságot és azt, ahogyan az adott axióma érvénybe lép.

```
# (u+v) kiszámítása
for i in range(len(vector1)):
    sum_vector.append(vector1[i] + vector2[i])

# c(u+v) kiszámítása
for i in range(len(sum_vector)):
    scalar_mult.append(sum_vector[i]*scalar)
sum_vector_mx=np.array(sum_vector)
scalar_mult_mx=np.array(scalar_mult)

# cu és cv kiszámítása
for i in range(len(vector1)):
    scalar_vector1.append(vector1[i] * scalar)
    scalar_vector2.append(vector2[i] * scalar)
scalar_vector1_mx = np.array(scalar_vector1)
scalar_vector2_mx = np.array(scalar_vector2)

# segédvektorok kiszámítása
for i in range(len(vector1)):
    vector1_into_sum.append(sum_vector_mx[i]-vector1[i])
    vector2_into_sum.append(sum_vector_mx[i]-vector2[i])
for i in range(len(vector1)):
    scalar1_into_scalar_mult.append(scalar_mult_mx[i]-scalar_vector1[i])
    scalar2_into_scalar_mult.append(scalar_mult_mx[i]-scalar_vector2[i])
```

5.23. ábra. Részlet a *vectorSpaceProblems* – *vector_space()* metódusából

5.5.1. Alterek

A lineáris algebrában egy vektortérnek egy nem üres U részhalmaza V részhalmazát akkor nevezzük a V vektortér alterének, ha U maga is vektortér ugyanazon a K test felett a V -ből örökölt összeadás és skalárral való szorzás műveleteire nézve. Matematikailag ezt a kapcsolatot $U \leq_K V$ jelöléssel fejezzük ki.

Egy részhalmazról úgy látható be a legkönnyebben, hogy altér, ha ellenőrizzük a műveletekre való zártágát. Ezt ahogyan a fenti vektortereknél is említettem, azt jelenti, hogy tetszőleges U -beli vektorok összege, valamint egy U -beli vektornak a testből vett tetszőleges skalárral vett szorzata szintén eleme kell legyen az U halmaznak. Ebből adódóan minden altérnek kötelezően tartalmaznia kell a vektortér nullvektorát is.

Minden vektortérnek léteznek úgynevezett triviális alterei, ami azt jelenti, hogy maga a teljes V vektortér, valamint kizárólag a nullvektorból álló halmaz mindig alteret alkot. Geometria megközelítésben például az $\mathbb{R}^3$ valós vektortérben az origón átmenő egyenesek és az origón átmenő síkok mind kiváló példái a nem triviális altereknek.

Ahogyan a lineáris kombinációnál már említettem, az alterek szerkezete szorosan összefügg a lineáris kombináció fogalmával, hiszen ha vesszük egy tetszőleges $v_1, v_2, \dots, v_n$ eleme U vektorokat, és hozzájuk $a_1, a_2, \dots, a_n$ eleme K együtthatókat, akkor az $a_1 v_1 + a_2 v_2 + \dots + a_n v_n$ alakban felírt összeget lineáris kombinációnak nevezzük.

Ha egy U vektorhalmaz összes lehetséges lineáris kombinációját képezzük, akkor megkapjuk az U halmaz burkát. Ha U altere V -nek, akkor az U burka is egy zárt alteret alkot. Ha a burok az egész vektorteret generálja, akkor generátorrendszeréről beszélünk.

Az alterek közötti kapcsolatot vizsgálva megállapítható, hogy a két altér metszete szintén mindig altér lesz az adott vektortérben. Ezzel szemben a két altér egyszerű uniója általában nem alkot alteret. Az alterek összevonására ezért az alterek összege fogalmát használjuk $U_1 + U_2$, ami a két altér elemeinek összes lehetséges összegeként előálló vektorok halmaza, és ez a legszűkebb altér, amely mindkét eredeti alteret magában foglalja.

A számítógépes környezetben ezt úgy valósítottam meg, hogy amikor a program megkapta a felhasználótól a két vektort, a sík generálásához két kétdimenziós paraméterrácsot (az S és T jelű mátrixokat) használtam – hiszen a cél egy háromdimenziós térben elhelyezkedő sík felületi prezentációja volt. Ezután lineáris kombinációval kiszámoltam a sík felületét alkotó X , Y , és Z pontokat, ami lényegében majd meghatározza a síkot. Ezt úgy számoltam ki, hogy a vektor adott X , Y és Z koordinátáit, beszoroztam az S és T mátrixok megfelelő skalár értékeivel, majd pedig összeadtam. Ebből az eredményből adódóan, valamint ennek a kirajzolásából

kiválóan látszik, hogy az origón áthalad a sík bármilyen 2 vektor megadása esetén. Ezt képletben, például az X pontot az alábbi képlet segítségével valósítottam meg:

$$S \begin{pmatrix} x_1 \\ y_1 \\ z_1 \end{pmatrix} + T \begin{pmatrix} x_2 \\ y_2 \\ z_2 \end{pmatrix} = \begin{pmatrix} Sx_1 + Tx_2 \\ Sy_1 + Ty_2 \\ Sz_1 + Tz_2 \end{pmatrix}, \quad (50)$$

amit az alábbi ábrán látható módon oldottam meg (5.24. ábra).

```
# két sík előkészítése
s = np.linspace(-1, stop: 1, num: 10)
t = np.linspace(-1, stop: 1, num: 10)

# síkká alakítás
S, T = np.meshgrid(s, t)

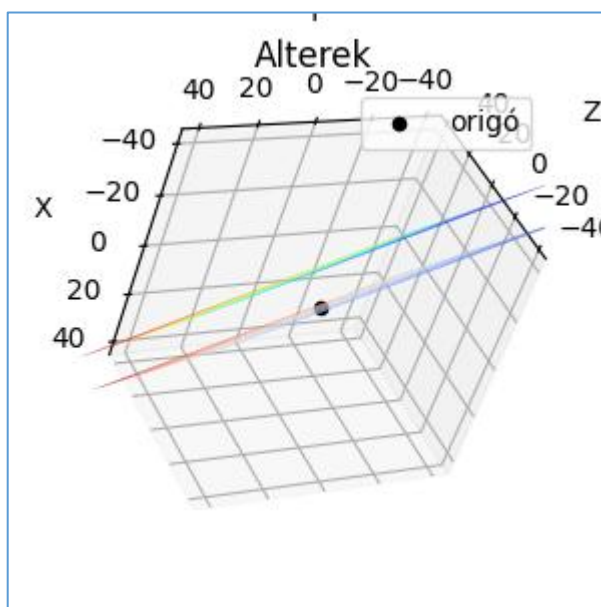
# az első sík koordinátáinak kiszámítása lineáris kombináció segítségével
X = vector_u[0] * S + vector_v[0] * T
Y = vector_u[1] * S + vector_v[1] * T
Z = vector_u[2] * S + vector_v[2] * T

# első sík kirajzolása (metszi az origót)
ax.plot_surface(X, Y, Z, alpha=0.9, cmap=plt.cm.coolwarm)

# második sík eltolása 20-al, többé már nem altér
Z2 = vector_u[2] * S + vector_v[2] * T + 20

# második sík kirajzolása a diagramra
ax.plot_surface(X, Y, Z2, alpha=0.9, cmap=plt.cm.jet)
```

5.24. ábra. Részlet a `vectorSubSpaces – show_diagram()` metódusából



5.25. ábra. Alterek diagrammja, elforgatva

5.5.2. Lineáris függetlenség

A vektortérben legyen $\{v_1, v_2, \dots, v_n\}$, $v_i \in V$ egy vektorrendszer. A $\{v_1, v_2, \dots, v_n\}$ vektorrendszert lineárisan függetlennek nevezzük, ha:

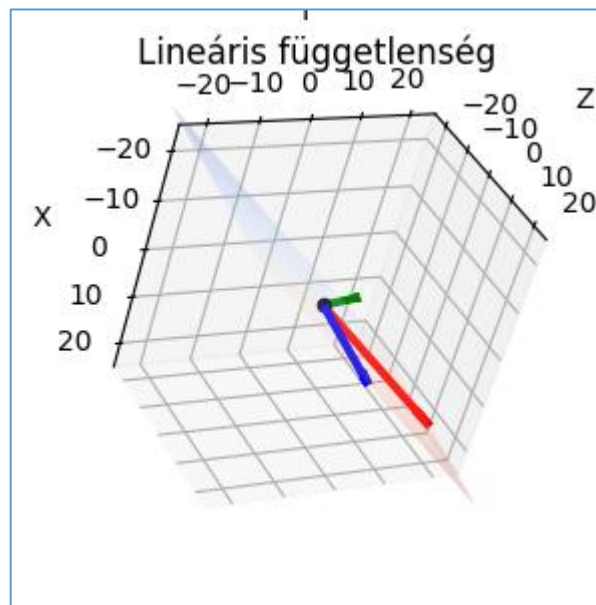
$$a_1 v_1 + a_2 v_2 + \dots + a_n v_n = 0_v \Rightarrow \alpha_1 = \alpha_2 = \alpha_n = 0. \quad (51)$$

Ha a rendszer nem lineárisan független akkor lineárisan függőnek nevezzük. Két vektor esetében, ha lineárisan függők, akkor egy egyenesre esnek. Ha függetlenek, akkor két különböző irányba mutatnak, és kifeszítenek egy síkot.

Három vektor esetében, ha lineárisan függők, akkor egy síkba esnek, azaz a harmadik nem mutat ki az első kettő által meghatározott síkból. Ha függetlenek, akkor kilépnek a harmadik dimenzióba.

Ez a háromdimenziós térben egy számítógépes környezetben kiválóan látható, hiszen a diagrammot a felhasználó kedve szerint forgathatja, és megvizsgálhatja, valamint láthatja, ahogyan a harmadik vektor lineárisan független, vagy függő lesz.

Az altereknél bemutatott (lásd 5.244. ábra.) megoldást használtam a sík létrehozására, valamint egyszerűen kirajzoltam a három megadott vektort a térben az origótól kiindulva. A halvány, többnyire átlátszó sík, és az első két megadott vektor kiválóan mutatja azt, hogy az első két vektorból készítettem a síkot. Ezután a harmadik vektort felrajzolva látszik, hogy a harmadik vektor lineárisan független, vagy függő.



5.26. ábra. Lineáris függetlenség 3 vektor esetében

Ahogy az előbb bemutatott ábrán is látható (5.26. ábra), a harmadik vektor elhagyja a két vektor által létrehozott síkot, és kilép a harmadik dimenzióba.

5.5.3. Bázis

A lineáris algebrában a bázis fogalma jelenti azt a szilárd alapot, amelyre a vektorterek egész elmélete épül. A bázis az egy lineárisan független generátorrendszer. Egy V vektortérben egy

$\bar{v}_1, \bar{v}_2, \dots, \bar{v}_n$ vektorrendszer akkor és csakis akkor alkot bázist, ha a tér tetszőleges v eleme egyértelműen felírható a bázisvektorok lineáris kombinációjaként, tehát az alábbi képlet szerint:

$$\bar{v} = a_1 v_1 + a_2 v_2 + \dots + a_n v_n. \quad (52)$$

Ahol az a_i együtthatók a $\bar{v}$ vektor egyértelműen meghatározott koordinátái. A rendszer lineáris függetlensége azt jelenti, hogy a zérusvektort kizárólag csupa nulla együtthatóval lehet előállítani:

$$\lambda_1 v_1 + \lambda_2 v_2 + \dots + \lambda_n v_n = 0 \Rightarrow \lambda_1 = \lambda_2 = \dots = \lambda_n = 0 \quad (53)$$

Amikor egy vektortér szerkezetét vizsgáljuk, kiderül hogy egy térnek végtelenül sok különböző bázisa lehet, de az összes létező bázisban a vektorok száma mindig pontosan ugyanannyi. Ezt a fix számot nevezzük a tér dimenziójának. A bázisok elemszámának állandóságát a kicserélési tételre alapozva lehet bebizonyítani. Ez a tétel azt mondja, hogy ha egy vektortérben n darab vektorból álló lineárisan független rendszerünk és egy k darab vektorból álló generátorrendszerünk van, akkor a lineáris rendszer elemeinek a száma sosem haladja meg a generátorrendszer elemeinek a számát:

$$n \leq k \quad (54)$$

A kicserélési tétel szerint legyen $f_1, \dots, f_n$ lineárisan független rendszer, és $g_1, \dots, g_k$ generátorrendszer egy V vektortérben. Ekkor bármely f_i -hez található olyan g_j , hogy

$$f_1, \dots, f_{i-1}, g_j, f_{i+1}, \dots, f_n \quad (55)$$

is lineárisan független rendszer.

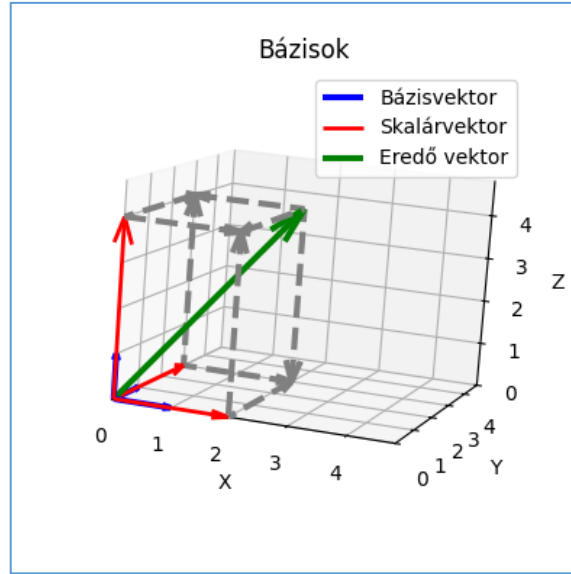
A bázis fogalma emellett elválaszthatatlan a koordinátázástól, és a vektorterek osztályozásától. Ha rögzítünk egy $B = \{b_1, b_2, \dots, b_n\}$ bázist, akkor a tér minden egyes $\bar{v}$ vektorához egyértelműen hozzárendelhető egy számoszlop, a koordináta-vektor:

$$[\bar{v}]_B = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}. \quad (56)$$

Ez az egyértelmű megfeleltetés hidat képez az elvont vektorterek és a numerikus számítások világa között. Ez a kapcsolat vezet el az izomorfizmus fogalmához is, ahol minden n -dimenziós K test feletti vektortér szerkezetileg teljesen azonos a K_n standard oszlopvektortérrel. Ez azt jelenti, hogy a bázis rögzítésével minden elvont vektortérbeli művelet visszavezethető egyszerű számtani műveletekre.

A számítógépes környezetben bemutatom, ahogyan egy háromdimenziós vektortérben adunk meg egy új koordináta-rendszert három bázisvektor segítségével, amiket egy mátrix oszlopaivá rendezünk. Ezután a bázisvektorokat az adott skalárokkal megszorozzuk. Az így

kapott elemek lineáris kombinációjaként előállítjuk az eredő vektort. Fontos megjegyezni, hogy csak akkor találtunk bázist, ha a felhasználó által kapott bázisvektorok által kapott mátrix determinánsa nem 0, valamint a mátrix rangja egyenlő 3-al.



5.27. ábra. Bázis diagramm

5.5.4. Bázis Transzformáció

Ha egy V vektortérben adott két bázis - a régi $B = \{b_1, b_2, \dots, b_n\}$, valamint az új $B' = \{b'_1, b'_2, \dots, b'_n\}$ - akkor az új bázisvektorokat felírhatjuk a régi bázisvektorok lineáris kombinációjaként:

$$b'_1 = t_{11}b_1 + t_{21}b_2 + \dots + t_{n1}b_n. \quad (57)$$

$\vdots$

$$b'_n = t_{1n}b_1 + t_{2n}b_2 + \dots + t_{nn}b_n. \quad (58)$$

Az így kapott együtthatókból képzett T mátrixot a B bázisból a B' bázisba való áttérési mátrixnak vagy transzformációs mátrixnak nevezzük. Ez a mátrix a bázisvektorok lineáris függetlensége miatt mindig reguláris, tehát a determinánsa nem nulla, így létezik inverze.

Ha egy tetszőleges $\bar{v}$ vektor koordinátái a régi bázisban $[\bar{v}]_B$ az új bázisban pedig $[\bar{v}]_{B'}$, akkor a kettő közötti kapcsolat:

$$[\bar{v}]_B = T \cdot [\bar{v}]_{B'} \quad (59)$$

$$[\bar{v}]_{B'} = T^{-1} \cdot [\bar{v}]_B \quad (60)$$

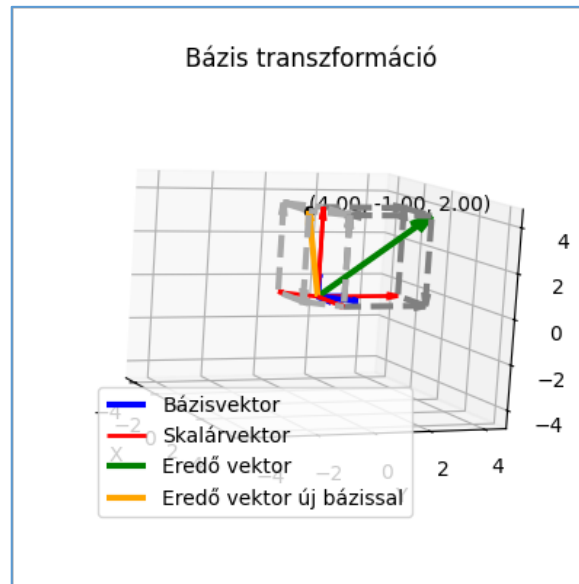
Amikor egy lineáris transzformáció mátrixát vizsgáljuk, akkor annak alakja erősen függ a választott bázistól. Ha a transzformáció mátrixa a B bázisban A , és a B' bázisban pedig D , és a két bázis közötti áttérési mátrix T , akkor a transzformáció új mátrixa a következő képlettel kapható meg:

$$D = T^{-1}AT \quad (61)$$

Azokra a mátrixokra, amelyekre fennáll a $B = T^{-1}AT$ összefüggés, hasonló mátrixoknak nevezzük. A hasonló mátrixok legfontosabb tulajdonsága, hogy a determinánsuk, rangjuk, nyomuk, és karakterisztikus polinomjuk megegyezik, mivel ugyanazt a lineáris transzformációt reprezentálják, csak más nézőpontból, tehát bázisból.

A gyakorlatban mindig olyan B' bázist keresünk, amelyben a transzformáció D mátrixa a lehető legegyszerűbb, ami ideális esetben diagonális tehát átlós alakú lesz, ami megkönnyíti a mátrixhatványozást és az egyéb számításokat.

Az alkalmazásomban egy háromdimenziós diagram segítségével kiválóan látható, hogy hogyan változnak meg egy adott térbeli pont koordinátái, ha egy új bázisvektorral fejezzük ki. Először a program kiszámolja az eredeti vektor felépítését az új mátrixban, majd pedig átszámítja az új bázisra egy gombnyomással.



5.28. ábra. Bázis transzformáció diagramm

5.5.5. Mátrix rangja

„Egy $B = \{v_1, v_2, \dots, v_n\}$ vektorrendszerrel maximálisan kiválasztható r lineárisan független vektorok számát a vektorrendszer rangjának nevezzük. egymásmellé helyezve a $v_1 \in R^m$ oszlopvektorokat egy $A = (v_1 v_2 \dots v_n) \in M_{mn}$ mátrixot kapunk, aminek a rangja nem más mint $\text{rang}(A)$ értelmezés szerint azonos az r vektorrendszerrel rangjával.

A mátrix rangjára a következő állítások igazak: $r \leq \min\{m, n\}$, $\text{rang}(A^t) = \text{rang}(A)$. A kicserélési tételből következik tehát, hogy a mátrix rangja egyenlő a maximálisan kiválasztható főelemek számával”. (Kupán, 2019)

Ezt a koncepciót szoros szálak fűzik a bázis és a dimenzió fogalmához, hiszen egy vektortér vagy altér bázisa magában foglalja a maximális lineáris függetlenséget, valamint a generált altér dimenziója megegyezik a bázisvektorok darabszámával. Egy másik fontos elméleti tétel kimondja, hogy a mátrix rangja megegyezik a hozzá tartozó lineáris transzformáció képterének a dimenziójával, ami a báziscserétől függetlenül állandó marad.

A mátrix rangjának gyakorlati kiszámítására a lineáris algebrában az egyik legelterjedtebb és legszisztematikusabb módszer a bázistranszformáció, amely lépésről lépésre haladva vizsgálja meg a vektorok egymástól való lineáris függetlenségét.

Először elkészítünk egy kiinduló táblázatot, amelyben a vizsgált vektorrendszer oszlopvektorait beírjuk a táblázat oszlopaiba. Ezután keresünk a táblázat belső mezőjében egy nullától különböző x_i elemet (főelemet). Ez az elem határozza meg, hogy melyik régi bázisvektort cseréljük le az új vizsgált vektorunkra. A transzformáció során a kiválasztott főelem sora és oszlopa mentén módosítjuk a táblázat többi elemét a báziscsere szabályai szerint. Az új táblázat elemeit a következő matematikai sémával számíthatjuk ki, ahol a főelem környezetében lévő értékeket vizsgáljuk:

$$\begin{array}{c|cc} \text{Bázis} & x & y \\ \hline e_i & x_i & y_i \\ e_j & x_j & y_j \end{array} \rightarrow \begin{array}{c|cc} \text{Bázis} & x & y \\ \hline x & 1 & \frac{y_i}{x_i} \\ e_j & 0 & \frac{x_i y_j - x_j y_i}{x_i} \end{array} \quad (62)$$

Ezt a lépést ismételjük mindaddig, amíg találunk bevihető vektort és egy nem nullával azonos főelemet. A rendszer rangja pontosan egyenlő azzal a számmal, ahány alkalommal sikeresen ki tudtunk választani főelemet.

Ha a folyamat végén a megmaradt szabad, tehát a be nem vitt vektorok oszlopaiban csupa nulla szerepel, az azt jelenti, hogy azok a vektorok a bázisba vett vektorok lineáris kombinációjaként állíthatók elő.

A számítógépes környezetben belül, amikor a felhasználó kitölti a neki tetszőleges $m \times n$ méretű mátrixot, a rendszer egy egyszerű számot visszaad, ami lényegében a kiszámolni kívánt mátrix rangja.

Mátrix		
15	19	46
11	50	35
25	50	26

Mátrix rangjának kiszámítása

A mátrix rangja: 3

5.29. ábra. Mátrix rangjának kiszámítása

5.5.6. Lineáris egyenletrendszerek

„Az általános lineáris egyenletrendszerek megoldására az egyik legtermészetesebben adódó, egyszerű és gyakorlati szempontból is jól alkalmazható eljárás a Gauss-féle kiküszöbölés, amelynek számos fontos elméleti következménye is van. Speciális egyenletrendszerekre vonatkozik a Cramer-szabály, amely a determinánsok segítségével ad képletet a megoldásra.” (Fraud, 1996)

Egy k egyenletből álló n ismeretlenes lineáris egyenletrendszer általános alakja

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &= \beta_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n &= \beta_2 \\ &\vdots \\ a_{k1}x_1 + a_{k2}x_2 + \dots + a_{kn}x_n &= \beta_k \end{aligned} \quad (63)$$

ahol az a_{ij} együtthatók, és a β_i konstansok egy T kommutatív test elemei. Az egyenletrendszerek száma és az ismeretlenek száma egymástól függetlenül tetszőleges lehet.

Az egyenletrendszer egy megoldásán T -beli elemek egy olyan $\gamma_1, \dots, \gamma_n$ sorozatát értjük, amelyeket a megfelelő x_i -k helyére beírva, valamennyi egyenletben egyenlőség teljesül, viszont egy egyenletrendszer lehet ellentmondásos, tehát nincs megoldása, lehet határozott, ahol egyértelműen oldható meg, tehát pontosan egy megoldás van. Ezen említetteken kívül lehet határozatlan is, ahol egynél több megoldásunk van.

Egy lineáris egyenletrendszer megoldásához azokra a kérdésekre keressük a választ, miszerint mi a feltétele annak, hogy egy egyenletrendszer megoldható legyen, hány megoldás van, hogyan lehet az összes megoldást megtalálni, valamint milyen módszerrel juthatunk el a megoldáshoz. Ezekre a kérdésekre a válasz nem más mint a Gauss-kiküszöbölés.

A módszer lényege, hogy az egyenletrendszert vele ekvivalens, tehát azonos megoldáshalmazú átalakításokkal olyan egyszerűbb alakra hozzuk, amelyből az ismeretlenek közvetlenül leolvashatók. A folyamat során az úgynevezett elemi ekvivalens átalakításokat alkalmazzuk a sorokon:

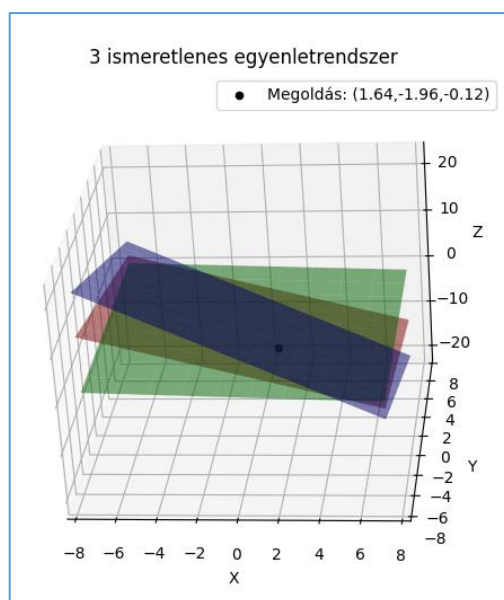
- E1. Valamelyik egyenletet egy nullától különböző skalárral végig szorozzuk.

- E2. Valamelyik egyenlethez egy másik egyenlet skalárszorosát hozzáadjuk.
- E3. Két egyenletet felcserélünk.
- E4. A tiszta nulla sorokat elhagyjuk.

Az elimináció során a főátló alatti elemeket lépésről lépésre kinullázzuk. Ezzel fokozatosan kiküszöböljük az $x_1, x_2, \dots, x_n$ ismeretleneket az alatta lévő egyenletekből.

Az említett Gauss-kiküszöbölés végén kapott vezéregyesek száma határozza meg a mátrix rangját. Ebből következik a megoldhatóság alapvető feltétele, miszerint az $Ax = b$ egyenletrendszer akkor és csak akkor oldható meg, ha az együtthatómátrix rangja megegyezik a kibővített mátrix rangjával. Ha a két rang nem egyezik meg akkor az egyenletrendszer ellentmondásos, tehát nincsen megoldása. Ha a két rang megegyezik és egyenlő az ismeretlenek számával, akkor a megoldás egyértelmű. Ha a két rang megegyezik viszont kisebb mint az ismeretlenek száma, akkor végtelen sok megoldás létezik, és az $n - r(A)$ darab szabadon megválasztható paraméterünk lesz.

Abban az esetben, ha az egyenletrendszerek száma megegyezik az ismeretlenek számával, valamint az A együtthatómátrix determinánsa nem nulla, akkor a megoldást Cramer-szabály segítségével is kiszámíthatjuk.



5.30. ábra. Háromismeretlenes lineáris egyenletrendszer megoldása

Kiválóan látható egy diagrammon ezen megoldások kirajzolása, hiszen ismerlentől függően egy kétdimenziós, vagy háromdimenziós diagrammra felrajzolva a megoldás pontjait, látható, ahogyan az egyenletek metszik egymást egy pontban, és ebben a metszéspontban helyezkedik el a pontos megoldás. Egy két ismeretlenes egyenletrendszerben ahol két egyenlet van, a két

egyenes ahol metszi egymást, ott helyezkedik el a megoldás. Ugyanígy egy három ismeretlenes egyenletrendszerben ahol három egyenletünk van, a három sík metszéspontjában helyezkedik el a megoldásunk.

5.6. Lineáris algebra a mindennapokban

A lineáris algebrát a mindennapi életben sok helyen használjuk, hiszen nagymennyiségű adat hatékony feldolgozását teszi lehetővé, ezek pedig mátrix műveletekkel egyszerűen elvégezhetőek. Számos területen alkalmazzák, például az egyik alkalmazási terület a digitális képfeldolgozás, hiszen minden digitális kép egy hatalmas mátrix ami a pixelek színértékeit tárolja, és ezeket manipulálva, különböző műveletekkel módosíthatunk, ezzel valósul meg a képszerkesztés. Fontos szerepe van a modern közgazdaság területén is, az alkalmazásban ezen területből egy egyszerű portfólió optimalizációs példát mutatok be, valamint az ezt követő fejezetekben bemutatok egy lineáris programozási példát is.

5.6.1. Digitális képfeldolgozás

A digitális képfeldolgozással mindennapi életünk során folyamatosan találkozunk, hiszen a digitális világ a napjaink részeivé vált, és minden pontján szerepel már. Egy digitális kép felbontástól függetlenül matematikailag egy mátrixot alkot. Ez a mátrix $m \times n$ mérettel rendelkezik, ami megegyezik a felbontással. Minden eleme egy-egy színkódot tárol, ami meghatározza az adott pixel színét.

Egy fekete-fehér képnél egy pixel 0 és 255 közötti értéket vehet fel, ami a teljes sötétségtől a maximális fényerőig terjed el. A kettő között a szürke különböző árnyalatai helyezkednek el.

A való életben a legtöbb kép amit készítünk színes. Mivel egyetlen 0 és 255 közötti értékkel nem lehet kifejezni a szivárvány minden színét, ezért három mátrixot rétegezzük egymásra, ez az RGB színmodell, ahol minden pixelhez három szín adat tartozik. R mint Red, tehát piros, hogy mennyire legyen piros az adott pixel. G mint Green, tehát zöld, mennyire legyen zöld, valamint B mint Blue, vagyis mennyire legyen kék az adott képpont. Összesen $256 \times 256 \times 256$ különböző színt vagyunk képesek kikeverni, ami körülbelül 16,7 milliárd szín.

$$\text{színes pixel} = \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (64)$$

Mivel egy kép egy mátrix, egyszerű alpműveletekkel képesek vagyunk a képet manipulálni, azaz szerkeszteni, ahogyan a felhasználó kívánja. Az alkalmazásban bemutatom, ahogyan egy képet képesek vagyunk negatívvá tenni, tükrözni vízszintesen és függőlegesen, RGB színeket hozzáadni a meglévő képhez, fekete-fehér képpé változtatni, valamint szaturációt változtatni



5.31. ábra. Digitális képfeldolgozás felülete

A képek szerkesztését a *PIL*, *NumPy*, és a *Tkinter*en belüli *filedialog* segítségével oldottam meg, ahol először a képet tallózva, tehát kiválasztva egy tetszőleges képet a fájlrendszerünkből, kieszadjuk az elérési útvonalát. A *PIL* segítségével ezt a képet megnyitjuk, és átkonvertáljuk RGB-be, hogy képesek legyünk részletesebben manipulálni a képünket.

Egy kép negatívvá tétele az egyik leglátványosabb, és legegyszerűbb művelet, hiszen csak annyi a dolgunk, hogy a maximális színértékből, tehát 255-ből kivonjuk az eredeti pixel értékét az egész mátrixra nézve.

```
# kép negatívvá tétele
1 usage
def negative_image(self):
    # manipulálhatóvá tenni a képet
    data = np.array(self.image)

    # kivonunk 255-ből az adott pixel értékét (az összes pixelre)
    negative_data = 255 - data

    # biztosítjuk, hogy a kép formátuma ne sérüljön (0-255 közötti számok)
    negative_img = Image.fromarray(negative_data.astype('uint8'))

    #kép megjelenítése az előnézethez
    self.original_image = negative_img
    self.display_img(self.original_image)
```

5.32. ábra. Kép negatívvá tétele

Egy képet képesek vagyunk tükrözni is függőlegesen és vízszintesen. Egy tükrözésnél legyen az vertikális vagy horizontális, a kép mátrixának a sorai vagy oszlopai művelettől függően, megmaradnak a helyükön, de a sorrendje teljesen megfordul.

```
# kép vízszintes tükrözése
horizontal_invert_data = np.flip(data,axis=1)
```

5.33. ábra. Vízszintes tükrözés

Mivel egy színes kép mátrixa rendelkezik RGB színcsatornákkal, ezért képesek vagyunk egyenként, külön-külön hozzáadni pluszban RGB értékeket, attól függően, hogy melyik csatornához mennyit szeretnénk hozzáadni. Ez alatt azt kell érteni, hogyha a felhasználó azt szeretné, hogy egy képet pirosabbá tesz, akkor a teljes képhez még adhat hozzá piros értéket. Ilyenkor a teljes kép mátrixának az RGB színcsatornák közül az R értékéhez hozzáadódik a kívánt érték, és látványosan pirosabb lesz az egész kép. Ez működik G és B-re is, valamint egyszerre több színnel is. Az alkalmazásban egyszerű csúszkák segítségével oldottam meg, aminek a minimum értéke 0 tehát nem kívánunk abból az értékből hozzáadni, a maximum pedig 255. Ettől függetlenül biztosítottam a túlsordulást, és átalakítottam *float32*-re. Ez azért szükséges, mert az alap *uint8* típusú képek maximális értéke 255 lehet, akkor a csatorna értéke átfordulna a skála elejére, tehát például 260-ból 4 lenne, ami azt jelenti, hogy egy nagyon világos pirosból nagyon sötét pixel lenne, és ez tönkre tenné a képet. Ezért átalakítva, elkerüljük ennek a veszélyét, és később egyszerűen ha túllépjük a 255-öt, akkor az érték továbbra is ehhez a maximumhoz tartson.

```
# RGB színcsatornákhoz szín hozzáadása
data[:, :, 0] += red
data[:, :, 1] += green
data[:, :, 2] += blue
```

5.34. ábra. RGB színcsatornákhoz szín hozzáadása

Egy képet fekete-fehérré az ITU-R BT.601 (ITU-R, 2011) szabvány alapján szürkítettem, hiszen az emberi szem különböző hullámhosszú fényekre eltérő érzékenységgű, ezért a szabvány alapú képlet segítségével egy természetesebb szürke képet hozhatunk létre. Szürkíthetnénk az RGB színek átlagolásával is, viszont azzal a módszerrel egy természetellenes végeredményt kapunk, ami az emberi szemnek nem annyira kellemes érzés. A luminancia módszerrel, az alábbi képlet alapján számolható ki:

$$Y' = 0.299 \cdot R' + 0.587 \cdot G' + 0.114 \cdot B'. \quad (65)$$

Miután készen van a szürkítés, ahhoz hogy a kész kép továbbra is szerkeszthető maradjon, hogyha például RGB színeket szeretnénk külön hozzáadni a képhez, akkor egyszerűen ezeket a szürke pixel értékeket egymásra pakoljuk egy háromelemű mátrixra, ahogyan egy RGB értéket tárol, ezután problémamentesen tudunk dolgozni az új képpel.

```

# szürkítés
1 usage
def gray_scaling(self):

    # kép manipulálhatóvá tétele
    data = np.array(self.original_image).astype(np.int16)

    # luminancia módszerrel szürkítés  $0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$ 
    gray_scale = 0.299 * data[:, :, 0] + 0.587 * data[:, :, 1] + 0.114 * data[:, :, 2]

    # a kapott szürkített értéket háromszorozzuk és egymásra pakoljuk RGB pixel formára
    grayed_data = np.stack([gray_scale] * 3, axis=-1)

    # visszaalakítjuk, levágjuk a határokon túli értékeket, és megjelenítjük
    new_data = np.clip(grayed_data, a_min: 0, a_max: 255).astype('uint8')
    new_img = Image.fromarray(new_data)

    self.display_img(new_img)

```

5.35. ábra. Kép szürkítése

Egy kép élénkségét úgy tudjuk kiszámolni, hogy vesszük a fekete-fehér mását. Ezután az eredeti színes, és a szürke kép közötti különbséget kiszámoljuk, ezt tiszta színinformációnak nevezzük. Ezután a telítettségi csúszkából kapott értéket, amit aszerint húzunk, hogy mennyire élénk színt szeretnénk magunknak, ezzel az értékkel megszorozzuk az előbb kiszámolt tiszta színinformációt, majd pedig hozzáadjuk az alap szürke képhez.

```

# élénkség állítása
1 usage
def saturation(self, sat_slider):

    # élénkség csúszkából kapott érték
    sat_value = float(sat_slider.get()) / 100

    # kép manipulálhatóvá tétele
    data = np.array(self.original_image).astype(np.float32)

    # luminancia módszerrel szürkítés  $0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$ 
    gray_base = data[:, :, 0] * 0.299 + 0.587 * data[:, :, 1] + 0.114 * data[:, :, 2]

    # a szürke képet manipulálhatóvá alakítjuk
    gray_rgb_layers = np.stack([gray_base] * 3, axis=-1)

    # az új élénkített képet kiszámoljuk  $gray\_value + sat\_value \cdot (original\_img - gray\_value)$ 
    saturated_data = gray_rgb_layers + sat_value * (data - gray_rgb_layers)

    # kép visszaalakítása és megjelenítése
    new_data = np.clip(saturated_data, a_min: 0, a_max: 255).astype('uint8')
    new_img = Image.fromarray(new_data)
    self.display_img(new_img)

```

5.36. ábra. Kép élénkségének változtatása

5.6.2. Lineáris programozás

A lineáris programozás az operációkutatás egyik legfontosabb ága, amely olyan feltételes szélsőérték-feladatok megoldásával foglalkozik, ahol mind a korlátozó feltételek, mind a célfüggvény lineárisak.

“Legyen egy $m \times n$ méretű mátrix, b egy m -komponensű vektor, és egy c n -komponensű vektor. Keresünk egy olyan n komponensű x vektort, amely maximalizálja a cx lineáris függvényt az $Ax = b, x \geq 0$ lineáris feltételek mellett:

$$cx \rightarrow \max \quad (66)$$

$$Ax = b, x \geq 0. \quad (67)$$

Ez a lineáris programozási feladat egységes, úgynevezett kanonikus alakban. Általános formájában egy LP feladat tartalmazhat egyenlőtlenségi feltételt, előjelkötetlen változókat, maximalizálás helyett lehet minimalizálás a cél. Az LP feladat különböző alakjai azonban ekvivalensek abban az értelemben, hogy bármely LP feladat kanonikus alakban felírható és fordítva, egy kanonikus alakban felírt LP feladat átalakítható más, például csupa egyenlőtlenségi feltételt tartalmazó feladattá.” (Komáromi, 2002)

A gyakorlati és gazdasági problémák modellezése során azonban ritkán találkozunk eleve ilyen tiszta struktúrával, hiszen a korlátok gyakran egyenlőtlenségként jelentkeznek, vagy a döntési változók előjele kötetlen. Ezen problémák mindegyike módszeresen visszavezethető a kanonikus alakra.

Amennyiben egy változó feltétel nem egyenlőségként, hanem például egy $\sum_{j=1}^n a_{ij}x_j \leq b_i$ formájú egyenlőtlenségként adott, úgy egy nem negatív u_i lazító változó hozzáadásával a feltétel $\sum_{j=1}^n a_{ij}x_j + u_i = b_i$ alakú egyenlőséggé alakítható, ahol az $u_i \geq 0$ feltétel a felhasználatlan erőforrás mértékét reprezentálja. Ha egy döntési változó tetszőleges valós értéket felvehet, azaz előjelkötetlen, akkor a matematikai formalizmus megtartása érdekében felírható két új, szigorúan nem negatív változó különbségeként, vagyis az $x_j = x'_j - x''_j$ helyettesítéssel, ahol $x'_j \geq 0$ és $x''_j \geq 0$. Valamint, mivel a minimalizálási és maximalizálási feladatok szoros kapcsolatban állnak egymással, a célfüggvény irányának megváltoztatása egyszerűen kezelhető azon azonosság révén, miszerint a cx függvény maximalizálása teljesen ekvivalens a $-cx$ függvényérték minimalizálásával.

A lehetséges megoldások $L = \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}$ halmaza konvex halmazt alkot, mivel lineáris egyenletek által meghatározott hipersíkok és zárt félterek metszeteiként jön létre. Egy halmaz akkor konvex, ha bármely két pontját összekötő szakasz minden pontját is

tartalmazza. A konvex halmazok szeparációs tétele kimondja, hogy ha egy pont nem eleme egy zárt, konvex halmaznak, akkor létezik egy elválasztó hipersík, amelyre igaz, hogy $cx > \alpha > c\hat{x}$ minden $x \in H$ esetén.

A dualitáselmélet alapja a Farkas-tétel, mely szerint az $Ax = b, x \geq 0$ és az $yA \geq 0, yb < 0$ alternatív rendszerek közül pontosan az egyik oldható meg. Minden kanonikus primál feladathoz hozzárendelhető egy duális feladat: $\min\{yb\}$, feltéve, hogy $yA \geq c$, ahol y változók a korlátok gazdasági árnyékait jelentik.

Mivel a lineáris programozási feladatok elmélete alapján az optimális megoldás mindig a lehetséges megoldások konvex halmazának egy szélső pontjában található, a keresés a véges számú bázismegoldásokra korlátozható. Dantzig Szimplex módszere egy táblázat, az úgynevezett Szimplex-tábla segítségével lépked a csúcsok között. A bázismegoldás optimális, ha a redukált költségek sorában minden elem nemnegatív. Ha egy oszlopban a relatív költség negatív, de az összes többi együttható nempozitív, a célfüggvény felülről nem korlátos, így nincs optimum.

Az alkalmazás a klasszikus lineáris programozási feladatok megoldására és vizualizációjára lett tervezve. Optimális érték kiszámítása után, két- akár háromdimenziós térben is ábrázolhatja a lehetséges megoldások terét, és a keresett optimumot. Először a felhasználó legenerálja a kívánt mezőket a korlátozási feltételek és ismeretlenek száma szerint. Ezután a program létrehozza a kitöltendő mezőket. A korlátozási feltételek mellett megjelenik a célfüggvény, valamint a minimum vagy maximum kiszámításához szükséges opciók.

A program a `scipy.optimize.linprog()` függvény segítségével határozza meg a keresett optimumot. A függvény alapértelmezetten minimalizálási problémát vár, ezért a feltételeket $\leq$ vagy $=$ formára kell hozni. Ezt úgy tudjuk elérni, hogy amikor a program egyenlőtlenségről egyenlőtlenségre nézi az előjeleket, és $\geq$ jelet talál, akkor egyszerűen mindkét oldalát megszorozza -1 -gyel, ami megfordítja az egyenlőtlenség irányát.

```
#megforditjuk az elojelt
elif(limit_type==">="):
    A_ub.append([value * -1 for value in current_row])
    b_ub.append(current_limit_val * -1)
```

5.37. ábra. Egyenlőtlenségek megfordítása $\geq$ esetén

Az egyenlőtlenségeket és egyenlőségeket külön válogatja, valamint a rendszer eldönti, hogy a változók nem-negatívak, nem-pozitívak vagy szabad változók-e. Miután a rendszer kiszámolta a megoldást, korrekciót végzünk, ha maximalizálási feladatról volt szó, hiszen a

függvény minimalizálási feladatot számol, és arra is alakítottuk az értékeket a számolás elején. Ezt követően ismeretlenek számától függően a rendszer a kellő diagrammot megjeleníti a kiszámolt optimummal együtt.

Korlátozási feltételek				
2	21	5	>=	38
30	42	35	>=	24
8	21	10	>=	9
x1	x2	x3	>=	7

Függvény

37 47 46

Minimum vagy Maximum számolás

☒ Minimum ☐ Maximum

Kiszámítás

5.38. ábra. Lineáris programozás interfész

5.6.3. Portfólió optimalizálás

Harry Markowitz 1952-es Portfolio Selection című úttörő munkája óta a modern portfólióelmélet alapaxiómája, hogy a befektetők kockázatkerülők. Ez azt jelenti, hogy törekednek a bizonytalanság és az ingadozás minimalizálására. A portfólió-optimalizálás egyik legfontosabb megközelítése, amikor a befektetési eszközök megfelelő súlyozásával olyan kombinációt hozunk létre, amely a diverzifikáció (Bélyácz, 2001) révén a lehető legkisebbre szorítja le az együttes kockázatot.

A portfólió teljes kockázatát a hozamok szórásával mérjük. Egy két eszközből álló portfólió varianciája a következő képlettel határozható meg:

$$\sigma_p^2 = w_1^2 \cdot \sigma_1^2 + w_2^2 \cdot \sigma_2^2 + 2 \cdot w_1 w_2 \sigma_{12}, \quad (68)$$

ahol w_1 az első eszköz súlya, w_2 a második eszköz súlya (aránya) a portfólióban. Mivel a teljes tőkét felosztjuk, a súlyok összege mindig 1. σ_1 és σ_2 az egyes értékpapírok saját egyedi kockázata, tehát szórása. Valamint σ_{12} a két értékpapír közötti kovariancia, amely azt méri, hogyan ingadozik a két eszköz egymáshoz képest.

A gyakorlatban a kovariancia kiszámításához a szemléletesebb korrelációs együtthatót használjuk:

$$\sigma_{12} = \rho_{12} \cdot \sigma_1 \cdot \sigma_2. \quad (69)$$

Ezt behelyettesítve a varianciafüggvénybe, majd abból négyzetgyököt vonva kapjuk meg a portfólió tényleges kockázatát, tehát szórását:

$$\sigma_p = \sqrt{w_1^2 \cdot \sigma_1^2 + w_2^2 \cdot \sigma_2^2 + 2 \cdot w_1 w_2 \sigma_{12}} \quad (70)$$

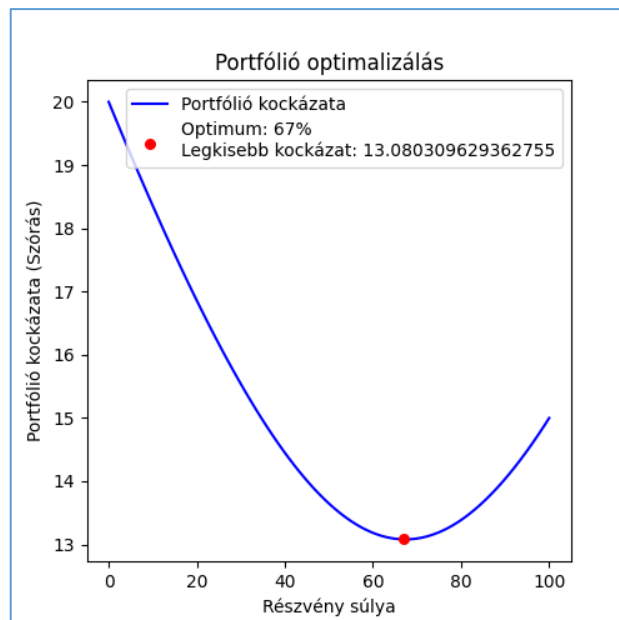
A kockázatsökkenés mértéke a korrelációs együtthatótól függ, amely -1 és $+1$ között mozoghat. Ha $\rho_{12} = +1$ akkor a két eszköz teljesen együtt mozog, a kockázat nem csökkenthető a súlyozással, az csupán az egyedi kockázatok számtani átlaga lesz. Ha $\rho_{12} < 1$ akkor a portfólió eredő kockázata alacsonyabb lesz, mint az egyedi kockázatok súlyozott átlaga. Minél közelebb van a korreláció a -1 -hez, a kockázat annál drasztikusabban csökkenthető.

A program fő célja az optimalizálás, amely azt jelenti, hogy megkeresi azt a pontot, ahol a két eszköz kombinációjából adódó szórásfüggvény eléri az abszolút minimumát. Létezik egy egzakt matematikai képlet, amely megadja a minimális kockázatú súlyt:

$$w_1^* = \frac{\sigma_2^2 - \rho_{12} \cdot \sigma_1 \cdot \sigma_2}{\sigma_1^2 + \sigma_2^2 - 2\rho_{12} \cdot \sigma_1 \cdot \sigma_2} \quad (71)$$

A program egy *for* ciklus segítségével egyesével kiszámolja az összes lehetséges százalékos súlyeloszlást, ezt lementi egy listába, majd pedig kiválasztja a legkisebb értéket, és a hozzá tartozó százalékos arányt.

A két megoldás ugyanazt a minimális kockázatú pontot találja meg a szórásgörbén, azért választottam a ciklussal való végigjárást, hogy a program egy diagram használatával látványosan ki tudja rajzolni, ahogyan a kockázat változik a százalékos arány változásával.



5.39. ábra. Minimális kockázatszámítás utáni diagram

6. Következtetések

Az alkalmazás fejlesztése során célom egy olyan digitális példatár létrehozása volt, amely egyszerű és interaktív módon elősegíti a lineáris algebrai problémák bemutatását és tanulmányozását. A dolgozatban bemutatott rendszer betekintést nyújt ezen problémák megismeréséhez: a tanulók olvashatnak a választott probléma szabályairól, ismertetőiről, ezeket kipróbálhatják és tanulhatnak belőle, a tanárok akár órán az egyszerű interfésznek köszönhetően könnyen és látványosan megértethetik az adott probléma működését.

A fejlesztés során külön figyelmet fordítottam az felhasználói élményre, gyors és optimalizált környezetre, könnyen megérthető szabályokra és leírásokra, valamint az átlátható és jól strukturált rendszerarchitektúrára.

Jövőbeli tervek közé sorolható a rendszer teljes mértékű átírása egy weboldalas környezetre, amit a felhasználók még egyszerűbben elérhetnek. A példatár bővíthető további problémák bemutatásával különböző fejezetekből a lineáris algebrán belül, valamint több mindennapi alkalmazással.

Összességében elmondható, hogy az alkalmazás hozzájárulhat a lineáris algebrai ismeretek bővítéséhez, és fejlesztéséhez. Hasznos eszközt kínálhat a jövőben mindazok számára, akik egyszerűen, hatékonyan, és látványos formában szeretnék az alkalmazás által bemutatott példákból tanulni, és fejlődni, gyors megoldásokat ellenőrizni, vagy vizualizálni.

Irodalomjegyzék

1. Bélyácz, I. (2001). *Befektetés-Elmélet*. Pécs: Pécsi tudományegyetem kiadó.
2. Fraud, R. (1996). *Lineáris algebra*. ELTE Eötvös kiadó.
3. *Geogebra*. (2026). Forrás: <https://www.geogebra.org/>
4. *Git hivatalos dokumentáció*. (2025). Forrás: <https://docs.github.com/en>
5. *Importlib dokumentáció*. (2025). Letöltés dátuma: 2025. 12. 18, forrás: <https://docs.python.org/3/library/importlib.html>
6. ITU-R. (2011). *Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios*. Genf, Svájc: International Telecommunication Union.
7. Komáromi, É. (2002). *Lineáris programozás*. Budapest: Budapesti Közgazdaságtudományi és Államigazgatási Egyetem, Operációkutatás Tanszék.
8. Kupán, P. (2019). *Lineáris algebra és alkalmazásai*. Kolozsvár: Scientia Kiadó.
9. *Matplotlib dokumentáció*. (2026). Letöltés dátuma: 2026. 1. 25., forrás: <https://matplotlib.org/stable/index.html>
10. *MVC dokumentáció*. (2025. 12. 10.). Forrás: <https://www.geeksforgeeks.org/software-engineering/mvc-framework-introduction/>
11. *NumPy dokumentáció*. (2026). Letöltés dátuma: 2026. 1. 9., forrás: <https://numpy.org/doc/stable/index.html>
12. *Pathlib dokumentáció*. (2025). Letöltés dátuma: 2025. 12. 18., forrás: <https://docs.python.org/3/library/pathlib.html>
13. *PIL dokumentáció*. (2026). Letöltés dátuma: 2026. 4. 17., forrás: <https://pillow.readthedocs.io/en/stable/>
14. Puskás, C., Szabó, I., & Tallos, P. (1997). *Lineáris algebra*. Budapest: Budapesti Közgazdaságtudományi Egyetem.
15. *Pycharm dokumentáció*. (2025). Letöltés dátuma: 2025. 12. 9., forrás: <https://www.jetbrains.com/help/pycharm/getting-started.html>
16. *Python dokumentáció*. (2025). Letöltés dátuma: 2025. 12. 10., forrás: <https://docs.python.org/3/>
17. *SciPy dokumentáció*. (2026). Letöltés dátuma: 2026. 4. 25., forrás: <https://docs.scipy.org/doc/scipy/>
18. *Symbolab*. (2026). Forrás: <https://www.symbolab.com>
19. *Tkinter dokumentáció*. (2025. 12. 15.). Forrás: <https://docs.python.org/3.11/library/tkinter.html>